

COSMIC Multi-Agent System — Architecture & Implementation Spec

Agent Runtime Contract v1.6

Layer	Technology
Transport	Redis Streams
Routing	Model Router (Groq classifier) — three routes: opus, gemini, perplexity
Session & Memory	Session Manager (Gateway) + Qdrant local + .md file store
Embeddings	Qwen3-embedding-8b via OpenRouter
Credentials	Gateway Credential Manager (OAuth PKCE, encrypted token store)
Process Management	supervisord (containers) / systemd (bare-metal)
Protocol	Agent Runtime Contract v1.6
Tool Access	Declared per agent in agent_card.yaml
Agent IDs	{org}/{name}:{version}

This is the single implementation-ready spec. All code in this document reflects the final v1.6 contract. There are no superseded sections, no version history, no delta patches. Build from this document only.

1. System Architecture: Three Layers

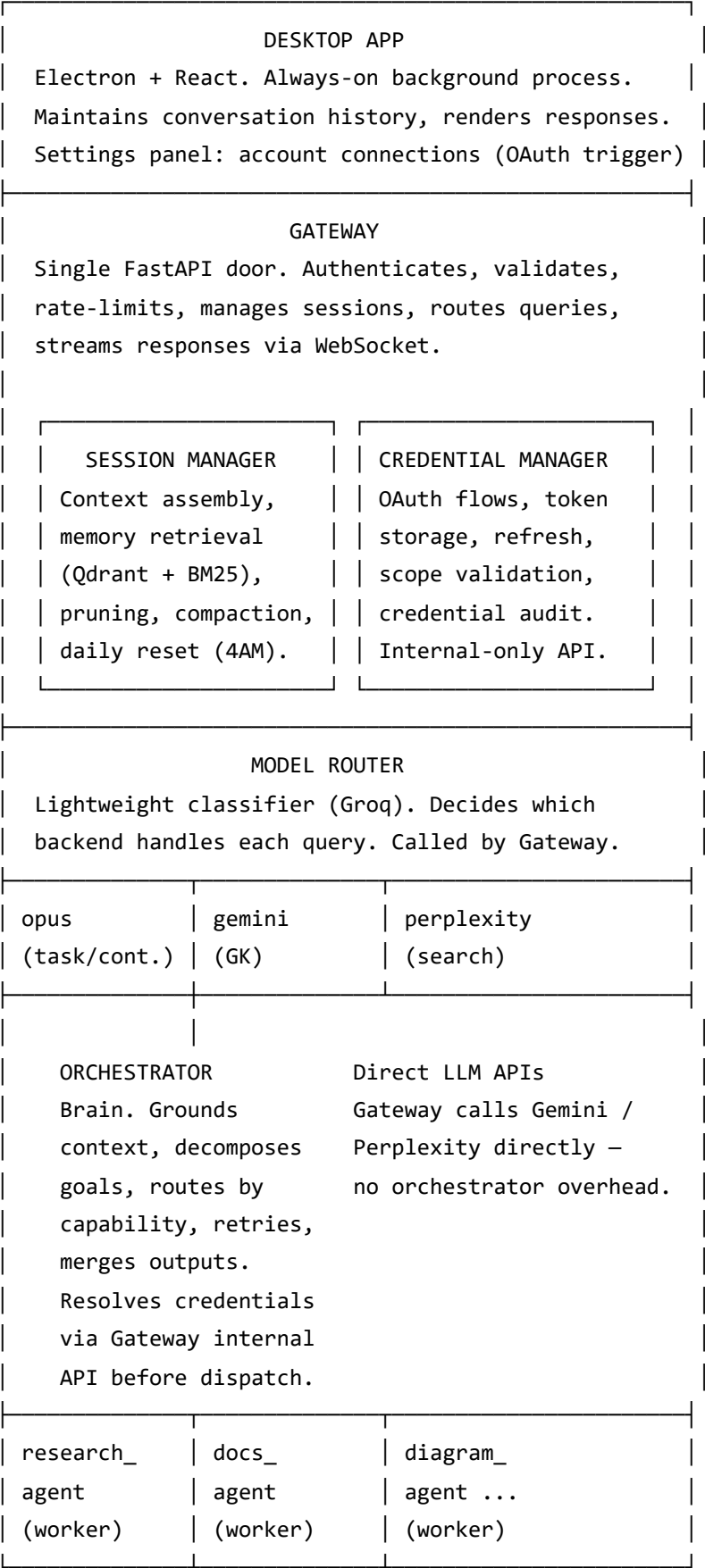
Every design decision flows from one mental model. Keep these three layers strictly separated — confusion comes from mixing them.

Layer	Question	Examples
Contract (WHAT moves)	Message schemas, envelope shapes	TaskEnvelope, EventEnvelope, Heartbeat

Layer	Question	Examples
Transport (HOW it moves)	Wire protocol, queue technology	Redis Streams
Topology (WHERE it runs)	Process layout, scaling model	supervisord workers, containers, K8s

The contract never changes across deployment targets. Only transport and topology evolve.

1.1 Layer Diagram

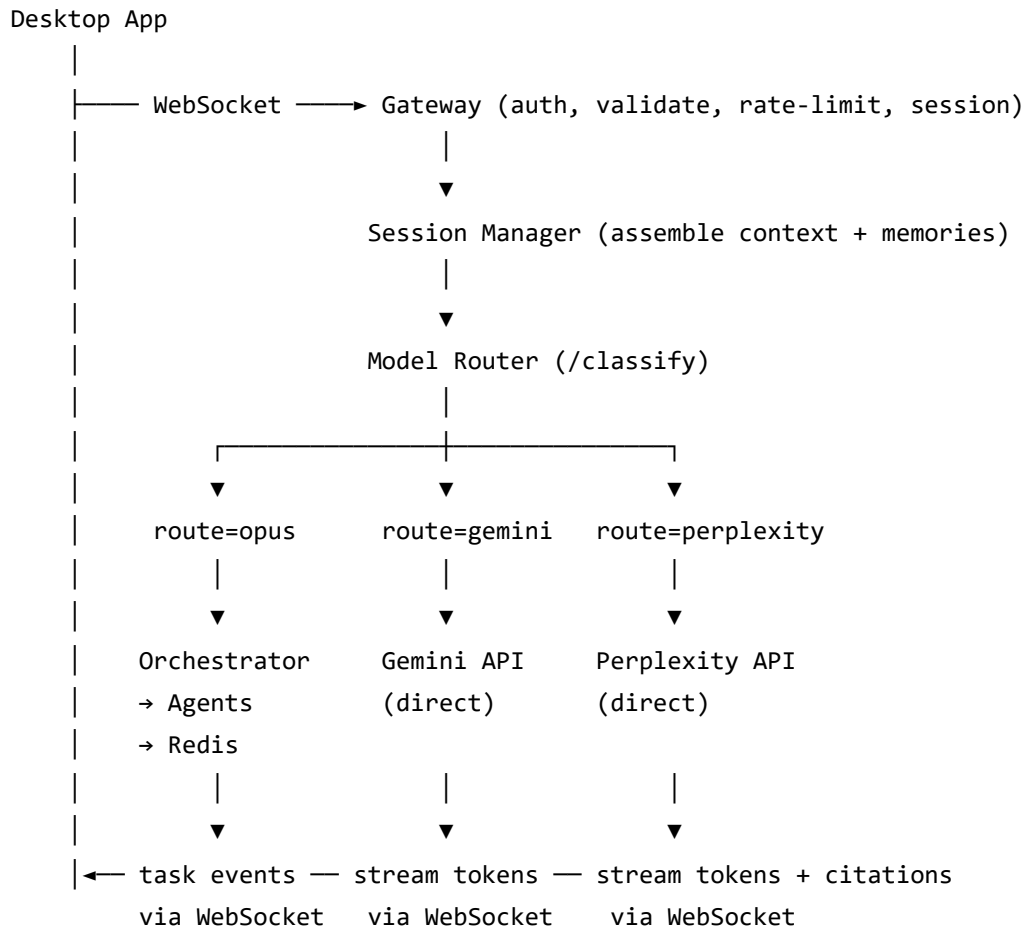


↑ All agents identical in shape ↑
Each follows Agent Runtime Contract v1.6

Component	Responsibility
Desktop App	Electron + React UI. Always-on background process registered at startup. Maintains conversation history locally. Connects to Gateway via WebSocket. Settings panel triggers OAuth account connections via Gateway.
Gateway	Receives requests from desktop app, validates auth + schema, assembles session context via the Session Manager (today's conversation + retrieved memories), calls Model Router for classification, routes to appropriate backend, streams responses and task events via WebSocket. Owns the Credential Manager (§22) and Session Manager (§23).
Model Router	Lightweight stateless classifier. Determines which backend handles a query: <code>opus</code> (orchestrator — tasks, continuations, ambiguous input), <code>gemini</code> (direct API), or <code>perplexity</code> (direct API). Called by Gateway after context assembly. No <code>unknown</code> route — <code>opus</code> is the fallback.
Orchestrator	Reads context, decomposes goals into subtasks, queries registry for capable healthy agents, resolves credentials via Gateway internal API when intents require provider access, dispatches via Redis, merges results. Powered by Claude Opus.
Sub-Agent Worker	Single-domain specialist. Consumes Task Envelopes from its Redis stream, emits Event Envelopes, writes artifacts, sends heartbeats

1.2 Request Flow

Every user query follows one of four paths based on the Model Router's classification:



Why three paths? A simple general knowledge question ("What is a knowledge graph?") does not need Claude Opus, agent decomposition, or Redis task envelopes. Routing it directly to Gemini saves cost and latency. Only genuine tasks ("Draft an email to my team"), continuations, and ambiguous inputs go through the orchestrator. There is no unknown route — the orchestrator handles anything the classifier can't confidently categorize. All three paths receive the same assembled session context (today's conversation + retrieved memories) from the Session Manager (see §23).

2. Model Router

The Model Router is a lightweight classification service that determines which backend should handle a user query. Its purpose is cost optimization — avoiding the expensive Opus orchestrator for queries that don't need it.

2.1 Classification Routes

Route	Backend	When	Cost
opus	Orchestrator (Claude Opus)	Tasks, continuations, ambiguous input, progress queries, anything referencing prior work	High
perplexity	Perplexity API (direct)	Real-time info, citations, current events, verification	Medium
gemini	Gemini API (direct)	General knowledge, explanations, theory, brainstorming	Low

2.2 Hard Routing Rules

These rules are enforced post-classification and are non-negotiable:

- 1. If `is_task` is true → `opus` (always, regardless of other flags)
- 2. If input is a continuation or ambiguous (`go on` , `ok` , `why?` , progress queries) → `opus` (the orchestrator is smart enough to handle these — it has full session context and task state)
- 3. Else if `needs_latest` OR `needs_citations` → `perplexity`
- 4. Else → `gemini`

There is no unknown route. A personal assistant should never bounce back "please clarify." The orchestrator (Opus) handles ambiguous inputs — it can respond conversationally, ask a specific clarifying question, or take action based on context. This is a UX decision: the smartest model handles the hardest-to-classify inputs.

```
def enforce_rules(parsed: dict) -> dict:
    out = {
        'route': parsed.get('route', 'gemini'),
        'needs_latest': bool(parsed.get('needs_latest')),
        'needs_citations': bool(parsed.get('needs_citations')),
        'is_task': bool(parsed.get('is_task')),
        'is_continuation': bool(parsed.get('is_continuation')),
        'confidence': parsed.get('confidence', 0.5),
        'signals': parsed.get('signals', []),
    }

    if out['is_task']:
        out['route'] = 'opus'
    elif out['is_continuation']:
        out['route'] = 'opus'
    elif out['needs_latest'] or out['needs_citations']:
        out['route'] = 'perplexity'
    elif out['confidence'] < 0.5:
        out['route'] = 'opus'          # low confidence → let Opus decide
    else:
        out['route'] = 'gemini'

    return out
```

2.3 Architecture

The Model Router runs as a standalone FastAPI microservice, called by the Gateway via internal HTTP after authentication.

Property	Value
Server	FastAPI + uvicorn
Port	8742 (internal, not exposed to desktop app)
Classifier model	openai/gpt-oss-20b via Groq API
Protocol	HTTP/2 with pre-warmed connection pool
Typical RTT	< 100ms

Why behind the Gateway, not before it? Placing the Model Router after the Gateway ensures:

- Unauthenticated requests never reach the classifier (no wasted Groq API calls)
- The Gateway owns all cross-cutting concerns (auth, rate limiting, sessions)

- The Model Router stays a pure stateless classifier with no auth/session logic
- The desktop app has a single endpoint to talk to (the Gateway)

2.4 Classification Endpoint

```
# POST http://localhost:8742/classify
# Request
{
  "query": "What are today's headlines about Nvidia?",
  "conversation_context": [                # optional
    {"role": "user", "content": "Tell me about AI stocks"},
    {"role": "assistant", "content": "Here are the top AI stocks..."}
  ],
  "max_completion_tokens": 380
}

# Response
{
  "classification": {
    "route": "perplexity",
    "needs_latest": true,
    "needs_citations": true,
    "is_task": false,
    "confidence": 0.92,
    "signals": ["current_events", "news_query", "time_sensitive"]
  },
  "metrics": {
    "rtt_ms": 67.3,
    "connection_warmed": true,
    "http2_enabled": true
  },
  "classifier_model": "openai/gpt-oss-20b",
  "raw_classifier_output": "{...}",
  "timestamp_unix_ms": 1706000000000
}
```

2.5 Conversation Context

The Model Router accepts optional conversation history for better classification. The Gateway attaches the last N messages from the session to the classification request, along with the route used for each message. The Model Router does not maintain any session state itself.

With context, the classifier can distinguish between a continuation of a task discussion (`opus`) vs a follow-up to a knowledge question (`gemini`). Without context, ambiguous inputs like "go on" default to `opus` — the orchestrator has session state and task context to handle them intelligently.

2.6 Fallback & Error Handling

If the Model Router is unreachable or returns an error, the Gateway falls back to `opus` . This is the safest default — the orchestrator can handle any query type, even if it's more expensive. The system degrades gracefully rather than failing.

```
# Gateway-side fallback
async def classify_query(query: str, context: list) -> dict:
    try:
        result = await http_client.post(
            f'{MODEL_ROUTER_URL}/classify',
            json={'query': query, 'conversation_context': context},
            timeout=MODEL_ROUTER_TIMEOUT_SEC,
        )
        return result.json()['classification']
    except Exception:
        return {'route': 'opus', 'confidence': 0.0, 'signals': ['router_fallback']}
```

2.7 Configuration

All secrets and configuration are loaded from environment variables.

```
# Model Router environment
GROQ_API_KEY=<secret>
CLASSIFIER_MODEL=openai/gpt-oss-20b
MODEL_ROUTER_HOST=0.0.0.0
MODEL_ROUTER_PORT=8742
HTTP2_ENABLED=true
CONNECTION_POOL_SIZE=10
KEEPALIVE_EXPIRY=30

# Gateway references to Model Router
MODEL_ROUTER_URL=http://localhost:8742
MODEL_ROUTER_TIMEOUT_SEC=3
MODEL_ROUTER_FALLBACK_ROUTE=opus          # also used when confidence < 0.5
```

3. Gateway

The Gateway is the single entry point for the desktop application. It handles authentication, request validation, model routing, session management, credential management, and response delivery. No external client communicates with the orchestrator or agents directly.

3.1 Responsibilities

Responsibility	Description
Authentication	Validates local API token on every request and WebSocket connection
Session & memory management	Owns the Session Manager: assembles context (today's conversation + retrieved memories) for every query before classification. Manages daily sessions, context window pruning, compaction, and memory retrieval via Qdrant + BM25 (see §23)
Model routing	Calls Model Router to classify each query
Task dispatch	For <code>opus</code> route: creates TaskEnvelope, dispatches to orchestrator via Redis
Direct LLM proxy	For <code>gemini</code> / <code>perplexity</code> routes: calls LLM API directly, streams response
Event streaming	Streams task progress events to desktop app via WebSocket
Rate limiting	Token-bucket rate limiting per session
Credential management	Owns the Credential Manager: handles OAuth PKCE flows, stores encrypted refresh tokens, manages connected accounts, validates scopes, refreshes access tokens, and exposes internal-only endpoints for the orchestrator to resolve credentials at dispatch time (see §22)

3.2 Connection Model: WebSocket + REST

The desktop app maintains a persistent WebSocket connection for real-time bidirectional communication. REST endpoints are available for stateless operations and health checks.

Why WebSocket as primary? The desktop app is always-on (background process, registered as startup app). A persistent WebSocket connection eliminates connection setup overhead on every query and enables the server to push task events, progress updates, and clarification requests without polling.

3.3 WebSocket Protocol

Connection: `ws://localhost:8080/ws?token=<local_api_token>`

Client → Server messages:

```
# Submit a new query
{
  "type": "query",
  "session_id": "sess_abc123",          # null for new session
  "content": "Write a Python script to parse logs",
  "conversation_context": [             # last N messages for model router
    {"role": "user", "content": "I need to analyze server logs"},
    {"role": "assistant", "content": "What format are the logs in?"},
  ],
  "request_id": "req_001"               # client-generated, for correlation
}

# Reply to a clarification request
{
  "type": "reply",
  "task_id": "tsk_abc123",
  "content": "Use source A",
  "request_id": "req_002"
}

# Cancel an in-progress task
{
  "type": "cancel",
  "task_id": "tsk_abc123",
  "request_id": "req_003"
}
```

Server → Client messages:

```
# Classification result (sent immediately after model router returns)
```

```
{  
  "type": "route_result",  
  "request_id": "req_001",  
  "route": "opus",  
  "classification": { ... }  
}
```

```
# Streaming tokens (gemini/perplexity direct response)
```

```
{  
  "type": "response.chunk",  
  "request_id": "req_001",  
  "session_id": "sess_abc123",  
  "content": "Here is a script that...",  
  "done": false  
}
```

```
# Final response (gemini/perplexity complete)
```

```
{  
  "type": "response.complete",  
  "request_id": "req_001",  
  "session_id": "sess_abc123",  
  "content": "Full response text...",  
  "route": "gemini",  
  "metrics": { "rtt_ms": 450 }  
}
```

```
# Task created (opus route – orchestrator pipeline started)
```

```
{  
  "type": "task.created",  
  "request_id": "req_001",  
  "task_id": "tsk_abc123",  
  "session_id": "sess_abc123"  
}
```

```
# Task progress event (forwarded from orchestrator/agents)
```

```
{  
  "type": "task.progress",  
  "task_id": "tsk_abc123",  
  "agent_id": "cosmic/research-agent:1.0.0",  
  "payload": { ... },  
  "seq": 3  
}
```

```
# Task completed
```

```

{
  "type": "task.completed",
  "task_id": "tsk_abc123",
  "result": { ... },
  "artifacts": [ ... ]
}

# Task failed
{
  "type": "task.failed",
  "task_id": "tsk_abc123",
  "error": { "code": "TIMEOUT", "message": "..." }
}

# Clarification needed (from orchestrator escalation – see §13.2)
{
  "type": "clarify",
  "task_id": "tsk_abc123",
  "question": "Found conflicting sources. Which to prioritize?",
  "options": ["source_a", "source_b"]
}

# Error
{
  "type": "error",
  "request_id": "req_001",
  "code": "RATE_LIMITED",
  "message": "Too many requests. Try again in 5 seconds."
}

```

3.4 REST Endpoints

Method	Path	Purpose
POST	/query	Submit query (returns task_id for opus, or streams response for gemini/perplexity)
GET	/tasks/{task_id}	Get task status and result
GET	/tasks/{task_id}/events	SSE stream of task events
POST	/tasks/{task_id}/reply	Reply to clarification request
GET	/sessions/{session_id}	Get session conversation history

Method	Path	Purpose
DELETE	/tasks/{task_id}	Cancel a task
GET	/health	Health check
GET	/health/ready	Readiness check (all dependencies up)
	Credential Management (desktop app)	
GET	/auth/connect/{provider}	Initiate OAuth PKCE flow — redirects user to provider consent screen
GET	/auth/callback/{provider}	OAuth callback — exchanges code for tokens, stores encrypted refresh token
GET	/auth/accounts	List all connected provider accounts for the current user
DELETE	/auth/accounts/{account_id}	Disconnect a provider account — revokes tokens, marks credential revoked
	Credential Management (internal — orchestrator only)	
POST	/internal/credentials/resolve	Resolve credential_ref → short-lived access token. Authenticated via internal service token (see §22)
GET	/internal/credentials/accounts/{provider}	List connected accounts for a provider. Used by orchestrator for account resolution.
POST	/internal/credentials/lookup-resource	Look up resource bindings (e.g., doc_id → account) for orchestrator account resolution

3.5 Authentication

For the desktop app (localhost), authentication uses a local API token generated on first setup and stored in the app's secure storage (Electron `safeStorage`).

```

import hmac
from fastapi import Request, HTTPException, WebSocket

LOCAL_API_TOKEN = os.environ['GATEWAY_LOCAL_TOKEN']

async def verify_auth(request: Request):
    token = request.headers.get('Authorization', '').replace('Bearer ', '')
    if not hmac.compare_digest(token, LOCAL_API_TOKEN):
        raise HTTPException(status_code=401, detail='Invalid token')

async def verify_ws_auth(websocket: WebSocket):
    token = websocket.query_params.get('token', '')
    if not hmac.compare_digest(token, LOCAL_API_TOKEN):
        await websocket.close(code=4001, reason='Invalid token')
        return False
    return True

```

Future mobile app: When a mobile client is added, the auth layer can be extended with JWT or OAuth2 without changing the Gateway's internal routing logic. The auth middleware is a pluggable concern.

Internal service authentication: The `/internal/*` credential endpoints are protected by a separate internal service token, not the desktop app's local API token. Only the orchestrator holds this token. The desktop app cannot access internal credential endpoints.

```

INTERNAL_SERVICE_TOKEN = os.environ['GATEWAY_INTERNAL_TOKEN']

async def verify_internal_auth(request: Request):
    token = request.headers.get('X-Internal-Token', '')
    if not hmac.compare_digest(token, INTERNAL_SERVICE_TOKEN):
        raise HTTPException(status_code=403, detail='Internal access only')

```

3.6 Rate Limiting

Token-bucket rate limiting per session. Prevents runaway loops in the desktop app from overwhelming the backend.

```

RATE_LIMIT_TOKENS = 60          # requests per window
RATE_LIMIT_WINDOW_SEC = 60      # window size
RATE_LIMIT_BURST = 10           # max burst above steady rate

```

3.7 Request Processing Flow

```
async def handle_query(session_id: str, content: str, context: list, ws: WebSocket):
    # 1. Assign session
    if not session_id:
        session_id = generate_session_id()

    # 2. Rate limit check
    if not rate_limiter.allow(session_id):
        await ws.send_json({'type': 'error', 'code': 'RATE_LIMITED', ...})
        return

    # 3. Assemble context via Session Manager (see §23)
    assembled_context = await session_manager.assemble_context(
        session_id, content, context
    )
    # assembled_context = {
    #     'memories': [...],          # retrieved + ranked memories (10-12k budget)
    #     'conversation': [...],      # today's messages (pruned to fit window)
    #     'compacted_summary': '...', # if mid-day compaction has occurred
    # }

    # 4. Classify via Model Router
    classification = await classify_query(content, context)
    route = classification['route']

    await ws.send_json({
        'type': 'route_result',
        'route': route,
        'classification': classification,
    })

    # 5. Route to appropriate backend (all receive assembled_context)
    if route == 'opus':
        await dispatch_to_orchestrator(session_id, content, assembled_context, ws)
    elif route == 'gemini':
        await stream_from_gemini(session_id, content, assembled_context, ws)
    elif route == 'perplexity':
        await stream_from_perplexity(session_id, content, assembled_context, ws)

    # 6. Store message in session history
    await store_message(session_id, 'user', content, route=route)
```


3.8 Direct LLM Routing (Non-Orchestrator Paths)

For `gemini` and `perplexity` routes, the Gateway acts as a streaming proxy. This avoids orchestrator overhead for simple questions.

```
class GeminiAdapter:
    """Streams responses from Google Gemini API."""
    async def stream(self, query: str, context: list, ws: WebSocket):
        async for chunk in self._call_gemini_streaming(query, context):
            await ws.send_json({
                'type': 'response.chunk',
                'content': chunk,
                'done': False,
            })
        await ws.send_json({'type': 'response.complete', 'route': 'gemini', ...})

class PerplexityAdapter:
    """Streams responses from Perplexity API with citations."""
    async def stream(self, query: str, context: list, ws: WebSocket):
        async for chunk in self._call_perplexity_streaming(query, context):
            await ws.send_json({
                'type': 'response.chunk',
                'content': chunk,
                'done': False,
            })
        await ws.send_json({'type': 'response.complete', 'route': 'perplexity', ...})
```

Each adapter:

- Streams response tokens to the desktop app via WebSocket
- Stores the complete response in session history after completion
- Handles API errors gracefully (retry with exponential backoff, then error to client)

3.9 Orchestrator Dispatch Path

For `opus` routes, the Gateway creates a `TaskEnvelope` and dispatches it to the orchestrator via Redis. It then consumes events from `streams:events` and forwards them to the desktop app in real time.

```

async def dispatch_to_orchestrator(session_id, content, context, ws):
    task_id = generate_task_id()
    task = TaskEnvelope(
        task_id=task_id,
        task_list_id=session_id,
        session_id=session_id,
        sender='cosmic/gateway:1.0.0',
        recipient='cosmic/orchestrator:1.0.0',
        intent='orchestrator.process',
        input={'query': content, 'conversation_context': context},
        idempotency_key=str(uuid4()),
        priority='normal',
        ...
    )
    await dispatch(task, redis)
    await ws.send_json({'type': 'task.created', 'task_id': task_id, ...})
    # Events are forwarded by the gateway event consumer (see §3.10)

```

Note: The Gateway dispatches to the orchestrator here. Credential resolution happens later — when the orchestrator decomposes the task into sub-tasks and dispatches to agents, it calls the Gateway's internal credential endpoint to resolve short-lived access tokens (see §22.3).

3.10 Gateway Event Consumer

The Gateway runs its own consumer group on `streams:events`, independent of the orchestrator's consumer group. This means both the Gateway and the orchestrator receive all events independently.

```

# On startup: create gateway consumer group
await redis.xgroup_create('streams:events', 'gateway', id='0', mkstream=True)

async def gateway_event_consumer():
    """Forwards task events to the correct WebSocket connection."""
    while True:
        events = await redis.xreadgroup(
            groupname='gateway',
            consumername=GATEWAY_INSTANCE_ID,
            streams={'streams:events': '>'},
            count=10,
            block=500,
        )
        for stream, messages in events:
            for msg_id, data in messages:
                event = EventEnvelope.model_validate_json(data['event'])
                ws = active_task_connections.get(event.task_id)
                if ws:
                    await ws.send_json(format_event_for_client(event))
                await redis.xack('streams:events', 'gateway', msg_id)

```

Two consumer groups on streams:events :

Consumer Group	Purpose
orchestrator	Routing logic, retry decisions, DLQ handling, deferred recovery
gateway	Forward events to desktop app via WebSocket

Both groups receive every event independently. Neither blocks the other.

3.11 Session Management

The Gateway maintains session state in SQLite. A single persistent "main" session per user provides the perpetual conversation feel. Daily sessions reset at 4AM with forced compaction. See §23 for the full Session & Memory Management specification.

```
-- gateway/sessions.db
CREATE TABLE sessions (
    session_id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL,
    created_at TIMESTAMP,
    updated_at TIMESTAMP,
    compaction_count INTEGER DEFAULT 0,    -- how many times compaction ran
    compacted_summary TEXT,               -- mid-day compaction summary (if any)
    metadata_json TEXT
);

CREATE TABLE messages (
    message_id TEXT PRIMARY KEY,
    session_id TEXT,
    role TEXT,                          -- 'user', 'assistant', 'system'
    content TEXT,
    route TEXT,                         -- 'opus', 'gemini', 'perplexity'
    task_id TEXT,                       -- null for non-opus messages
    created_at TIMESTAMP,
    FOREIGN KEY (session_id) REFERENCES sessions(session_id)
);

CREATE INDEX idx_messages_session ON messages(session_id, created_at);
```

3.12 Health & Readiness

```
@app.get('/health')
async def health():
    return {'status': 'healthy'}

@app.get('/health/ready')
async def readiness():
    checks = {
        'redis': await check_redis(),
        'model_router': await check_model_router(),
        'sqlite': check_sqlite(),
    }
    all_ok = all(v == 'ok' for v in checks.values())
    return {'ready': all_ok, 'checks': checks}
```

4. Agent ID Design

Agent IDs follow the `{org}/{name}:{version}` pattern (same as HuggingFace, Docker Hub, model registries).

4.1 Format

```
cosmic/research-agent:1.0.0
cosmic/docs-agent:2.1.0
cosmic/orchestrator:1.0.0

# External LLM models referenced inside agents
anthropic/claude-sonnet-4-6
ollama/qwen2.5:7b
```

4.2 Agent ID vs Display Name

Field	Value / Rule
agent_id	cosmic/research-agent:1.0.0
display_name	Research Agent
Purpose of ID	Machine identifier — used in code, DB, Redis streams, logs, artifact manifests. Never changes after creation.
Purpose of Name	Human-readable label for UI and docs. Can be rebranded freely without touching code.
Foreign key rule	agent_id is the FK everywhere. Task envelopes, event envelopes, heartbeats, artifacts — all reference agent_id string only.

Why version in the ID? Lets you run `research-agent:1.0.0` and `research-agent:2.0.0` side-by-side during canary rollout. Orchestrator routing config pins exact versions — no silent breaking changes from `latest` . Audit logs are unambiguous: you always know exactly which version produced an artifact.

4.3 Pydantic Definition

```
class AgentID(BaseModel):  
    org: str          # 'cosmic', 'anthropic'  
    name: str         # 'research-agent', 'docs-agent'  
    version: str      # semver '1.0.0' or date '2024-11-20'  
  
    def __str__(self):  
        return f'{self.org}/{self.name}:{self.version}'
```

5. Folder Structure

Every agent has the same shape on disk. This makes orchestrator tooling fully generic — it never needs custom logic per agent.

5.1 Top-Level Project Layout

```

└─ cosmic-agents/
├─ shared/
│   │ # Shared contracts (single source of truth)
│   │ └─ contracts.py
│   │   │ # All Pydantic envelope models
│   │   └─ registry.py
│   │     │ # Agent registry client
│   │     └─ redis_bus.py
│   │       │ # Redis Streams transport layer
│   │       └─ redis_client.py
│   │         │ # Redis client factory (decode_responses=True)
│   │         └─ auth.py
│   │           │ # HMAC signing + verification
│   │           └─ time_utils.py
│   │             │ # UTC datetime normalization
│   │             └─ events.py
│   │               │ # Atomic seq allocation
│   │               └─ idempotency.py
│   │                 │ # Idempotency enforcement
│   │                 └─ leader.py
│   │                   │ # Leader election + fencing
│   │                   └─ artifact_security.py
│   │                     │ # Path allowlist + integrity checks
│   │                     └─ config.py
│   │                       │ # TTL tunables, constants
│   │
│   └─ agents/
│     │ # One folder per agent
│     │ └─ orchestrator/
│     │   └─ research_agent/
│     │     └─ docs_agent/
│     │       └─ _template/
│     │         │ # Copy this to create a new agent
│     │
│   └─ gateway/
│     │ # FastAPI gateway service
│     │ └─ main.py
│     │   │ # Gateway entry point + WebSocket handler
│     │   └─ routes/
│     │     │ # REST endpoint modules
│     │     └─ adapters/
│     │       │ # Direct LLM adapters (Gemini, Perplexity)
│     │       │ └─ gemini.py
│     │       │   └─ perplexity.py
│     │       └─ credentials/
│     │         │ # Credential Manager module (see §22)
│     │         │ └─ manager.py
│     │         │   │ # Core credential logic (resolve, refresh, revoke)
│     │         │   └─ oauth.py
│     │         │     │ # OAuth PKCE flow handlers per provider
│     │         │     └─ providers.py
│     │         │       │ # Provider adapter registry (Google, GitHub, etc.)
│     │         │       └─ encryption.py
│     │         │         │ # Token encryption/decryption (Fernet envelope)
│     │         │         └─ routes.py
│     │         │           │ # OAuth + internal credential API routes
│     │         └─ session/
│     │           │ # Session Manager module (see §23)
│     │           │ └─ manager.py
│     │           │   │ # Core session logic (assemble context, prune, compact)
│     │           │   └─ memory_retriever.py
│     │           │     │ # Hybrid retrieval (Qdrant vector + BM25), ranking, fusion
│     │           │     └─ compaction.py
│     │           │       │ # Conversation compaction logic (summarization)
│     │           │       └─ memory_writer.py
│     │           │         │ # Write memories to .md files + embed into Qdrant
│     │           └─ sessions.py
│     │             │ # Session storage (SQLite)
│     │             └─ sessions.db
│     │               │ # Session + message storage
│     │               └─ credentials.db
│     │                 │ # Credential store (accounts, tokens, audit – see §22)
│     │
│   └─ memory/
│     │ # Persistent memory store (.md source of truth)
│     │ └─ sessions/
│     │   │ # Compacted daily session summaries

```

```

| | └─ 2025-01-15.md
| | └─ 2025-01-16.md
| └─ agent_notes/          # Agent-saved memories (synced from agents/*/store/)
| | └─ research_agent/
| | | └─ learnings.md
| | └─ docs_agent/
| |   └─ learnings.md
| └─ user_data/            # Indexed user data (emails, files, etc.)
| | └─ emails/
| |   └─ files/
| └─ tasks/                # Task result summaries (retrievable, not in main session)
|   └─ <task_id>.md
|
└─ qdrant_data/            # Qdrant local storage (vector index – rebuilt from memory/)
|
└─ model_router/          # Model Router classifier service
  └─ main.py               # FastAPI server (port 8742)
  └─ classifier.py         # Classification logic + rule enforcement
  └─ config.py             # Groq API config, thresholds
|
└─ registry/              # SQLite agent registry
  └─ registry.db
|
└─ runs/                  # Runtime artifacts (gitignored)
  └─ artifacts/
    └─ <task_id>/
|
└─ supervisord.conf        # Process management
└─ docker-compose.yml      # Container deployment
  # NOTE: agents/*/store/ directories MUST be mapped to persistent
  # volumes in docker-compose.yml. runtime/ directories are ephemeral.
  # gateway/credentials.db MUST also be on a persistent volume –
  # it contains encrypted refresh tokens and account bindings.
  # memory/ and qdrant_data/ MUST be on persistent volumes –
  # they contain the system's long-term memory.

```


5.2 Per-Agent Folder (Every Agent Is Identical in Shape)

```
agents/research_agent/
├─ agent_card.yaml          # WHO I am + WHAT I can do (capability declaration)
├─ __main__.py             # Entry point: python -m agents.research_agent
├─ agent.py                # Core agent logic
├─
├─ prompts/                # READ-ONLY at runtime. Version controlled.
│   └─ system.md           # System prompt (what this agent is)
│       └─ policies.md     # Rules, constraints, tool usage policies
├─
├─ skills/                 #
│   └─ SKILLS.md           # Domain knowledge, techniques, reference
├─
├─ schemas/                # Machine-readable API contracts
│   └─ intents/            # JSON schema per intent
│       └─ research.topic.input.json
│       └─ research.topic.output.json
│       └─ research.recall_session.input.json
│   └─ events/             # Event payload schemas
│       └─ task.accepted.json
│       └─ artifact.added.json
├─
├─ store/                  # PERSISTENT. Survives restarts. Backed up.
│   └─ learnings.md        # Accumulated agent knowledge & facts (see §12)
│       └─ data/           # Agent-managed storage – schema is agent's own
│           └─ (agent defines its own DB, files, indexes here)
├─
└─ runtime/                # EPHEMERAL. Gitignored. Recreated on restart.
    └─ state.db            # In-flight task state (suspension serialization)
    └─ cache/              # Cached fetches, embeddings
    └─ logs/               # Agent-level structured logs
```

5.3 Critical Runtime Rules

- `prompts/` and `skills/` → **READ-ONLY** at runtime. Agent cannot modify its own instructions.
- `schemas/` → **READ-ONLY** at runtime. Only changed via versioned deploys.
- `store/` → **PERSISTENT**. Survives restarts. Must be on a persistent volume in containerized deployments. Backed up.
- `store/learnings.md` → Agent's accumulated knowledge and facts, persisted across sessions. Updated by the agent after tasks. Each agent owns its own learnings — no cross-agent reads.
- `store/data/` → **Agent-managed storage**. Each agent defines its own database schema, session tables, indexes, and data files here. No uniform schema is imposed — a research agent might track

source credibility scores while a docs agent tracks edit history. Must be on a persistent volume.

- runtime/ → **EPHEMERAL**. Gitignored. Recreated on restart. Used for in-flight state, caches, and logs only.
- runs/artifacts/<task_id>/ → **PER-TASK isolation**. Never shared across tasks.
- Orchestrator **NEVER** silently rewrites prompts live. Use versioned rollout + rollback.
- Track sha256 hashes of prompt files in registry so audits detect drift.

6. Agent Card (agent_card.yaml)

The Agent Card is the machine-readable capability declaration registered at startup. The orchestrator reads it to populate the registry.

Agent Card vs Model Card: An Agent Card is written for machines — it describes the agent's runtime interface (intents, schemas, SLA, health contract). A Model Card is written for humans — it describes the underlying AI model (evals, training data, limitations). The orchestrator only reads Agent Cards.

6.1 Full agent_card.yaml

```
# agents/research_agent/agent_card.yaml
agent_id: cosmic/research-agent:1.0.0
display_name: Research Agent
description: >
  Specialist agent for web research, document fetching,
  citation extraction, and image discovery.

intents:
  - name: research.topic
    description: Deep-research a topic and return citations + summary
    input_schema: schemas/intents/research.topic.input.json
    output_schema: schemas/intents/research.topic.output.json
    timeout_sec: 180

  - name: research.find_image
    description: Find a relevant image for a given query
    input_schema: schemas/intents/research.find_image.input.json
    output_schema: schemas/intents/research.find_image.output.json
    timeout_sec: 60

  - name: research.recall_session
    description: Recall what happened in a previous session for this agent
    input_schema: schemas/intents/research.recall_session.input.json
    output_schema: schemas/intents/research.recall_session.output.json
    timeout_sec: 30

# No auth_requirements – research agent uses its own tools (web_search,
# web_fetch), not user OAuth credentials. Only declare auth_requirements
# for intents that call external provider APIs on behalf of the user.

artifact_types:
  - web_page
  - pdf
  - image
  - citation_pack

policies:
  network_access: true
  writable_paths:
    - runs/artifacts/research_agent
    - agents/research_agent/store
    - agents/research_agent/runtime
  tool_access:
```

```

- web_search
- web_fetch
- file_write
allowed_senders:
- cosmic/orchestrator:1.0.0
intent_authorization:
  research.topic: [cosmic/orchestrator:1.0.0]
  research.find_image: [cosmic/orchestrator:1.0.0]
  research.recall_session: [cosmic/orchestrator:1.0.0]

sla:
  max_concurrency: 4
  heartbeat_interval_sec: 10
  heartbeat_ttl_sec: 30
  max_task_duration_sec: 180    # longest possible task – used for XAUTOCLAIM tuning
  health_endpoint: /health
  retry_policy:
    max_attempts: 3
    backoff: exponential
    backoff_base_sec: 2          # 2s, 4s, 8s
    backoff_max_sec: 60
    retryable_codes:
      - TIMEOUT
      - NETWORK_ERROR
      - RATE_LIMITED
    non_retryable_codes:
      - INVALID_INPUT
      - AUTH_ERROR
      - SCHEMA_VIOLATION

stream_key: streams:cosmic/research-agent:1.0.0

version_info:
  semver: 1.0.0
  released_at: 2025-01-01
  deprecated_at: null
  remove_after: null
  changelog: CHANGELOG.md

```

Tool access is declared per agent under `policies.tool_access`. Agents can invoke tools via any mechanism — MCP servers, direct SDK calls, REST clients, shell commands — the contract only cares *what* tools are permitted, not *how* they're called.

Session recall intents like `research.recall_session` allow the orchestrator to query agents about past sessions. For example, when a user asks "explain the last edits we did on this doc," the orchestrator sends

a `docs.recall_session` task to the docs agent, which queries its own `store/data/` storage and returns the relevant history. Each agent is the authority on its own past work.

6.2 `auth_requirements` — Provider Credential Declaration

Agents that call external provider APIs on behalf of the user declare `auth_requirements` per intent. The orchestrator reads this at dispatch time to determine whether credential resolution is needed (see §22.3).

```
# agents/docs_agent/agent_card.yaml (excerpt)
```

```
agent_id: cosmic/docs-agent:2.1.0
```

```
display_name: Docs Agent
```

```
intents:
```

```
- name: docs.edit
```

```
  description: Edit a Google Doc section
```

```
  input_schema: schemas/intents/docs.edit.input.json
```

```
  output_schema: schemas/intents/docs.edit.output.json
```

```
  timeout_sec: 120
```

```
- name: docs.create
```

```
  description: Create a new Google Doc
```

```
  input_schema: schemas/intents/docs.create.input.json
```

```
  output_schema: schemas/intents/docs.create.output.json
```

```
  timeout_sec: 60
```

```
- name: docs.resolve_resource
```

```
  description: Search for a document by name across connected accounts
```

```
  input_schema: schemas/intents/docs.resolve_resource.input.json
```

```
  output_schema: schemas/intents/docs.resolve_resource.output.json
```

```
  timeout_sec: 30
```

```
- name: docs.recall_session
```

```
  description: Recall what happened in a previous session
```

```
  input_schema: schemas/intents/docs.recall_session.input.json
```

```
  output_schema: schemas/intents/docs.recall_session.output.json
```

```
  timeout_sec: 30
```

```
auth_requirements:
```

```
  docs.edit:
```

```
    provider: google
```

```
    scopes:
```

```
      - https://www.googleapis.com/auth/documents
```

```
  docs.create:
```

```
    provider: google
```

```
    scopes:
```

```
      - https://www.googleapis.com/auth/documents
```

```
      - https://www.googleapis.com/auth/drive.file
```

```
  docs.resolve_resource:
```

```
    provider: google
```

```
    scopes:
```

```
      - https://www.googleapis.com/auth/drive.metadata.readonly
```

```
# docs.recall_session has no auth_requirements – it only reads
# from the agent's own store/data/, no provider API calls needed.
```

Rules:

- Only intents that call external provider APIs declare `auth_requirements` .
- Intents that only use agent-local tools or read from `store/` do not declare them.
- The orchestrator checks `auth_requirements` before dispatch. If present, it resolves a `credential_ref` and short-lived access token via the Gateway's internal credential endpoint and includes them in `TaskEnvelope.input.auth` (see §7.3, §22.3).
- If the required scopes are not granted by the user's connected account, the orchestrator escalates via `user.input_required` asking the user to re-consent with the needed scopes.

7. Agent Runtime Contract v1.6

Seven models every agent speaks. Defined once in `shared/contracts.py` . Transport-agnostic — the same shapes work whether moving through Redis Streams or HTTP.

7.1 Shared Time Utilities

All datetime operations use these helpers. **Never use** `datetime.utcnow()` **anywhere** — it is deprecated in Python 3.12 and produces naive datetimes that crash when mixed with timezone-aware values from APIs.

```

# shared/time_utils.py
from datetime import datetime, timezone

def utcnow() -> datetime:
    """Always returns a timezone-aware UTC datetime."""
    return datetime.now(tz=timezone.utc)

def to_utc_aware(dt: datetime) -> datetime:
    """Normalize any datetime to timezone-aware UTC.
    - Already aware → convert to UTC.
    - Naive → assume UTC, attach tzinfo.
    """
    if dt.tzinfo is None:
        return dt.replace(tzinfo=timezone.utc)
    return dt.astimezone(timezone.utc)

def deadline_remaining_sec(deadline_ts: datetime) -> float:
    """Safe deadline arithmetic – normalizes both sides before subtracting."""
    return (to_utc_aware(deadline_ts) - utcnow()).total_seconds()

```

7.2 Redis Client Configuration

All Redis connections use `decode_responses=True` to ensure consistent string handling throughout the system. No manual `.decode()` calls anywhere.

```

# shared/redis_client.py
import redis.asyncio as redis

async def get_redis() -> redis.Redis:
    return redis.Redis(
        host=os.environ.get('REDIS_HOST', 'localhost'),
        port=int(os.environ.get('REDIS_PORT', 6379)),
        decode_responses=True,      # all reads return str, not bytes
    )

```

7.3 TaskEnvelope (Orchestrator → Agent)

How the orchestrator asks for work. Always created by orchestrator, never by agents directly.


```

from pydantic import field_validator

class TaskEnvelope(BaseModel):
    task_id: str # 'tsk_abc123'
    task_list_id: str # Groups related tasks
    parent_task_id: str | None # For subtask trees
    session_id: str | None # Links task to session context
    sender: str # 'cosmic/orchestrator:1.0.0'
    recipient: str # 'cosmic/research-agent:1.0.0'
    intent: str # 'research.topic'
    input: dict # Intent-specific payload
    input_artifacts: list[ArtifactManifest] = [] # Files passed TO the agent
    idempotency_key: str # UUID – required, enforced via SETNX
    deadline_ts: datetime | None # Timezone-aware or naive UTC
    priority: Literal['high', 'normal', 'low'] = 'normal'
    leader_epoch: int | None = None # Fencing token set by orchestrator
    signature: str # HMAC-SHA256 of canonical payload
    contract_version: str = CURRENT_WRITE_VERSION # dynamic – see §21
    created_at: datetime = Field(default_factory=utcnow)

    @field_validator('contract_version')
    @classmethod
    def check_contract_version(cls, v: str) -> str:
        """Validates on deserialization (inbound from streams). Accepts any
        version in the read window. Outbound validation uses
        validate_outbound_version() at dispatch time – see §21.

        NOTE: load_task_envelope() uses model_construct() to bypass this
        validator when loading from storage, because stored envelopes may
        predate the current acceptance window. prepare_for_redispatch()
        stamps CURRENT_WRITE_VERSION before dispatch – which passes both
        this validator and validate_outbound_version()."""
        if v not in ACCEPTED_READ_VERSIONS:
            raise ValueError(
                f'Unacceptable contract version: {v}. '
                f'Accepted: {ACCEPTED_READ_VERSIONS}'
            )
        return v

```

Field	Notes
task_id	Unique task identifier. Format: tsk_abc123
task_list_id	Groups related tasks in a session

Field	Notes
parent_task_id	For subtask trees — links child to parent
session_id	Links task to session context. Agents use this to load/store session data in their own store/data/
input_artifacts	Typed file references passed TO the agent — verified by sha256 before read
idempotency_key	UUID. Enforced via SETNX before every execution (see §14)
leader_epoch	Fencing token. Agents reject tasks from stale epochs (see §16)
signature	HMAC-SHA256 of full canonical payload. Stripped before passing to agent logic (see §15)
contract_version	Set from CURRENT_WRITE_VERSION config — not a hardcoded literal. Validated on read by validate_incoming_version() (see §21)

input.auth Convention (Credential Passthrough)

When an intent has `auth_requirements` declared in `agent_card.yaml` (see §6.2), the orchestrator resolves credentials at dispatch time and places them in `input.auth`. This is a convention on the `input: dict` field, not a separate model field.

```
# Example: TaskEnvelope.input for a docs.edit task
{
  'query': 'Add a conclusion section to the project proposal',
  'doc_id': 'abc123',
  'auth': {
    'credential_ref': 'cred_9f3a...',      # opaque reference for audit
    'access_token': 'ya29.a0AfH6SM...',   # short-lived (5-15 min)
    'provider': 'google',
    'scopes': ['https://www.googleapis.com/auth/documents'],
    'expires_at': '2025-01-15T10:05:00+00:00',
  },
}
```

Hard rules for `input.auth`:

- **Never** included in EventEnvelopes, artifacts, logs, `store/`, `runtime/`, or `learnings.md`.
- The agent base class extracts `input.auth` before passing `input` to the agent's `execute` method and exposes it via `self.auth` — a runtime-only context that is never serialized (see §12.6).
- `input.auth` **is** covered by the HMAC signature (it is part of `input: dict`), ensuring it cannot be tampered with in transit.

- If `input.auth` is absent, the agent must not attempt any provider API calls requiring user credentials.
- If the access token expires mid-task, the agent suspends and requests a refresh via `orchestrator.refresh_credential` reverse task (see §13.1, §22.5).

7.4 EventEnvelope (Agent → Orchestrator)

How agents report progress. Agents emit these asynchronously as work proceeds.

```
class EventEnvelope(BaseModel):
    task_id: str
    agent_id: str # who emitted this
    event_type: Literal[
        'task.accepted', # agent received and validated task
        'task.progress', # intermediate progress update
        'task.suspended', # agent waiting for reverse-task reply
        'task.resumed', # agent continuing after suspension
        'task.deferred', # NON-TERMINAL: another instance executing – orchestrator owns r
        'artifact.added', # artifact produced, manifest in payload
        'task.completed', # TERMINAL: success – closes task state
        'task.failed', # TERMINAL: failure – closes task state
        'task.dlq', # TERMINAL: dead letter queue – closes task state
        'task.rejected', # NON-TERMINAL: stale epoch – triggers redrive
    ]
    seq: int # monotonic PER-TASK via redis.incr()
    payload: dict # event-type-specific data
    emitted_at: datetime
    contract_version: str = CURRENT_WRITE_VERSION

    @field_validator('contract_version')
    @classmethod
    def check_contract_version(cls, v: str) -> str:
        """Inbound validation – accepts any version in the read window."""
        if v not in ACCEPTED_READ_VERSIONS:
            raise ValueError(
                f'Unacceptable contract version: {v}. '
                f'Accepted: {ACCEPTED_READ_VERSIONS}'
            )
        return v
```

Terminal vs Non-Terminal events:

```

TERMINAL_EVENTS = {'task.completed', 'task.failed', 'task.dlq'}
NON_TERMINAL_EVENTS = {'task.accepted', 'task.progress', 'task.suspended',
                        'task.resumed', 'task.deferred', 'artifact.added',
                        'task.rejected'}

```

- **TERMINAL** events expire the seq key, close task state in consumers.
- **NON-TERMINAL** events leave the seq key alive and task state open.
- `task.rejected` is explicitly **NON-TERMINAL** — the task will be redriven by the orchestrator with a current epoch.

seq is per-task, not per-agent. It resets to 1 for every new `task_id`. Sort by `(task_id, seq)` to reconstruct exact execution order during replay or debugging.

7.5 AgentResult (Returned on `task.completed` or `task.failed`)

```

class AgentResult(BaseModel):
    status: Literal['completed', 'failed']
    output: dict
    artifacts: list[ArtifactManifest]
    error: AgentError | None

class AgentError(BaseModel):
    code: str # 'TIMEOUT', 'NETWORK_ERROR', 'DEADLINE_EXCEEDED'
    retryable: bool
    message: str
    next_action: str | None # 'retry', 'escalate', 'skip'

```

7.6 TaskInProgress (Idempotency Sentinel)

Returned when another instance is currently executing this task. The orchestrator understands this as "check back later" — not an error, not a requeue.

```

class TaskInProgress(BaseModel):
    task_id: str
    idempotency_key: str
    executing_since: datetime
    check_after_sec: int # orchestrator should retry lookup after this delay

# Union return type for idempotency enforcement
ExecutionResult = AgentResult | TaskInProgress

```

7.7 Heartbeat (Agent → Registry, every N seconds)

```
class Heartbeat(BaseModel):
    agent_id: str
    instance_id: str          # unique per worker process
    healthy: bool
    current_load: int         # active tasks right now
    max_concurrency: int
    heartbeat_ttl_sec: int    # self-describing – used by router
    last_seen: datetime
```

Router only dispatches to agents where `healthy == True` , `current_load < max_concurrency` , and `last_seen` is within `heartbeat_ttl_sec` .

7.8 ArtifactManifest (What Was Produced)

```
class ArtifactManifest(BaseModel):
    artifact_id: str          # 'art_xyz789'
    task_id: str
    mime: str                 # 'application/pdf', 'image/png'
    sha256: str               # integrity hash
    path: str                 # 'runs/artifacts/tsk_abc123/paper.pdf'
    source_url: str | None
    created_by_agent: str     # agent_id
    created_at: datetime
```

8. Redis Streams: Queue Design

Redis Streams provide consumer groups, acknowledgement, replay, and crash recovery without adding a new service. Priority is implemented via stream tiers.

8.1 Stream & Key Naming

```
# Task streams: PER-AGENT (shared across all instances via consumer group)
streams:cosmic/research-agent:1.0.0:high
streams:cosmic/research-agent:1.0.0:normal
streams:cosmic/research-agent:1.0.0:low

# Liveness keys: PER-INSTANCE (each worker writes independently)
registry:cosmic/research-agent:1.0.0:research-1
registry:cosmic/research-agent:1.0.0:research-2

# Intent index: Redis Set mapping intent → agent_ids
intent:research.topic          → {cosmic/research-agent:1.0.0}
intent:research.find_image     → {cosmic/research-agent:1.0.0}

# Shared event stream – orchestrator AND gateway listen here
streams:events

# Orchestrator's own input streams (for reverse tasks from agents)
streams:cosmic/orchestrator:1.0.0:high
streams:cosmic/orchestrator:1.0.0:normal

# Dead letter queue
streams:dlq

# Capability updates (push-based discovery)
streams:capability.updates

# Idempotency keys
idempotency:{idempotency_key}      # execution lock
idempotency:result:{idempotency_key} # stored terminal result

# Seq counters
event_seq:{task_id}                # atomic per-task sequence allocation

# Leader election
orchestrator:leader                 # current leader instance_id
orchestrator:epoch                  # monotonic fencing token
```

Critical distinction: Liveness keys are per-instance (each worker reports its own load independently). Stream keys are per-agent (all workers consume from the same stream via a shared consumer group). Redis distributes messages across consumers in the group and tracks pending-per-consumer for crash recovery.

8.2 Consumer Group Setup

```
async def join_consumer_group(agent_id: str, instance_id: str, redis):
    for priority in ['high', 'normal', 'low']:
        stream = f'streams:{agent_id}:{priority}'
        try:
            await redis.xgroup_create(
                stream, 'workers',
                id='0',          # start from beginning
                mkstream=True    # create stream if not exists
            )
        except ResponseError as e:
            if 'BUSYGROUP' not in str(e):
                raise            # group already exists – expected, ignore
```

8.3 Dispatch (Orchestrator → Redis)

```
# shared/redis_bus.py

def load_task_envelope(task_id: str) -> TaskEnvelope:
    """Load a stored envelope from the orchestrator's task ledger.
    Uses model_construct() to bypass contract_version validation –
    stored envelopes may predate the current acceptance window.
    prepare_for_redispatch() will stamp CURRENT_WRITE_VERSION before dispatch."""
    row = db.execute(
        'SELECT envelope_json FROM tasks WHERE task_id = ?', [task_id]
    ).fetchone()
    data = json.loads(row['envelope_json'])
    return TaskEnvelope.model_construct(**data)

def prepare_for_redispatch(
    task: TaskEnvelope, new_epoch: int, secrets: dict
) -> TaskEnvelope:
    """Up-convert an older in-flight envelope to current write version.
    Called before every redispatch – recover_orphaned, task.rejected redrive,
    and deferred recovery. Preserves idempotency_key (no re-execution risk).
    MUST re-sign: contract_version and leader_epoch are covered by the HMAC,
    so mutating them without re-signing will fail verify_incoming()."""
    task.contract_version = CURRENT_WRITE_VERSION
    task.leader_epoch = new_epoch
    task.signature = sign_task(task, secrets[task.recipient])
    return task

async def dispatch(task: TaskEnvelope, redis: Redis):
    validate_outbound_version(task)    # enforce write-version at dispatch
    stream = f'streams:{task.recipient}:{task.priority}'

    # Backpressure check (see §8.5)
    length = await redis.xlen(stream)
    if length > STREAM_MAXLEN:
        raise BackpressureError(
            f'Stream {stream} at capacity ({length}/{STREAM_MAXLEN})'
        )

    await redis.xadd(
        stream,
        {'envelope': task.model_dump_json()},
        maxlen=STREAM_MAXLEN_APPROX,    # approximate trim as safety net
    )
```


8.4 Worker Loop (Agent Consuming from Redis)

```
STREAMS = {
    'high': f'streams:{AGENT_ID}:high',
    'normal': f'streams:{AGENT_ID}:normal',
    'low': f'streams:{AGENT_ID}:low',
}

# XAUTOCLAIM min_idle_time MUST exceed max_task_duration_sec from agent_card.yaml.
# If your longest intent is 180s, set this to at least 360s (2x).
# Too low → healthy workers mid-task get their messages reclaimed → duplicate pressure.
CLAIM_MIN_IDLE_MS = self.max_task_duration_sec * 2 * 1000

async def run(self):
    while True:
        # Crash recovery: reclaim messages from dead consumers
        for priority in ['high', 'normal', 'low']:
            claimed = await redis.xautoclaim(
                STREAMS[priority], 'workers', self.instance_id,
                min_idle_time=CLAIM_MIN_IDLE_MS,
                start_id='0-0'
            )
            if claimed[1]:
                for msg in claimed[1]:
                    await self._process_message(msg, STREAMS[priority])

        # Normal consumption: priority ordering with aging (see §18)
        consume_order = await self._priority_order_with_aging()
        for priority in consume_order:
            messages = await redis.xreadgroup(
                groupname='workers',
                consumername=self.instance_id,
                streams={STREAMS[priority]: '>'},
                count=1,
                block=100,          # ms – fall through to next tier
            )
            if messages:
                task = TaskEnvelope.model_validate_json(
                    messages[0]['envelope']
                )
                await self.handle(task, messages[0].id, STREAMS[priority])
                break                # restart priority loop after handling
```

Crash recovery is free. If a worker crashes mid-task, the message stays unacknowledged. On restart, XAUTOCLAIM reclaims stale messages. After N failed reclaims, the message moves to streams:dlq for manual inspection.

XAUTOCLAIM tuning rule: min_idle_time must be **at least 2x** the agent's max_task_duration_sec from its agent_card.yaml . If research.topic has timeout_sec: 180 , set min_idle_time to at least 360000 ms. Otherwise a healthy worker running a long task will have its message reclaimed, creating duplicate processing pressure.

8.5 Backpressure

If the orchestrator dispatches faster than agents consume, Redis streams grow unbounded. Backpressure prevents this.

```
# shared/config.py
STREAM_MAXLEN = 10000          # hard cap – dispatch rejects above this
STREAM_MAXLEN_APPROX = 12000  # approximate trim on XADD (allows brief overshoot)
```

Two-layer protection:

Layer	Mechanism	Behavior
Dispatch-time check	XLEN before XADD	If stream length exceeds STREAM_MAXLEN , dispatch raises BackpressureError . Orchestrator retries after backoff.
XADD approximate trim	maxlen=STREAM_MAXLEN_APPROX on every XADD	Safety net — Redis trims the stream approximately to this length. Uses ~ (approximate) to avoid O(N) trim on every write.

Gateway surfaces backpressure as HTTP 503 (Service Unavailable) with a Retry-After header. The desktop app shows a "system busy" state and retries automatically.

```
# Orchestrator handles BackpressureError
try:
    await dispatch(task, redis)
except BackpressureError:
    logger.warning(f'Backpressure on {task.recipient}, retrying after delay')
    await asyncio.sleep(backoff_sec)
# Re-attempt or queue internally
```

9. Process Management: supervisord

supervisord manages agent worker processes. Each agent is a supervised program that restarts automatically on crash.

Tool	When to Use
supervisord	User-space. Simple INI config. Natural fit inside Docker containers. Install via pip.
systemd	OS-level init. Linux only. Better for bare-metal/VM. Journald log integration. Use on COSMIC lab Dell servers.
Rule	Container → supervisord. Bare-metal lab server → systemd. Never mix within one deployment target.

9.1 supervisord.conf

```
[supervisord]
nodaemon=true
logfile=/var/log/supervisord.log

[program:gateway]
command=uvicorn gateway.main:app --host 0.0.0.0 --port 8080
autostart=true
autorestart=true
environment=GATEWAY_INTERNAL_TOKEN='<internal-service-token>',CREDENTIAL_ENCRYPTION_KEY='<fernet-key>'
stderr_logfile=/var/log/gateway.err.log
stdout_logfile=/var/log/gateway.out.log

[program:qdrant]
command=qdrant --storage-path ./qdrant_data
autostart=true
autorestart=true
stderr_logfile=/var/log/qdrant.err.log
stdout_logfile=/var/log/qdrant.out.log

[program:model_router]
command=uvicorn model_router.main:app --host 0.0.0.0 --port 8742
autostart=true
autorestart=true
environment=GROQ_API_KEY='<groq-api-key>',CLASSIFIER_MODEL='openai/gpt-oss-20b'
stderr_logfile=/var/log/model_router.err.log
stdout_logfile=/var/log/model_router.out.log

[program:orchestrator]
command=python -m agents.orchestrator
autostart=true
autorestart=true
environment=INSTANCE_ID='orchestrator-1',AGENT_SECRETS='{"cosmic/research-agent:1.0.0": "...", "cosm'
stderr_logfile=/var/log/orchestrator.err.log
stdout_logfile=/var/log/orchestrator.out.log

[program:research_agent_1]
command=python -m agents.research_agent
autostart=true
autorestart=true
environment=INSTANCE_ID='research-1',AGENT_SECRET='<research-agent-specific-secret>'
stderr_logfile=/var/log/research_agent_1.err.log
stdout_logfile=/var/log/research_agent_1.out.log
```

```
[program:research_agent_2]
command=python -m agents.research_agent
autostart=true
autorestart=true
environment=INSTANCE_ID='research-2',AGENT_SECRET='<research-agent-specific-secret>'
stderr_logfile=/var/log/research_agent_2.err.log
stdout_logfile=/var/log/research_agent_2.out.log
```

```
[program:docs_agent]
command=python -m agents.docs_agent
autostart=true
autorestart=true
environment=INSTANCE_ID='docs-1',AGENT_SECRET='<docs-agent-specific-secret>'
stderr_logfile=/var/log/docs_agent.err.log
stdout_logfile=/var/log/docs_agent.out.log
```

9.2 __main__.py Pattern (Every Agent)

```
# agents/research_agent/__main__.py
import asyncio
from .agent import ResearchAgent
from shared.redis_client import get_redis

async def main():
    redis = await get_redis()
    agent = ResearchAgent(redis=redis)
    await agent.register()
    await agent.run()

if __name__ == '__main__':
    asyncio.run(main())
```

10. Orchestrator Routing Config

Pin exact agent versions in routing config. Never use `latest` in production — it is a silent breaking change waiting to happen.

```
# routing.yaml – orchestrator reads this at startup
intents:
  research.topic:
    agent: cosmic/research-agent:1.0.0
    priority: normal
    fallback: null
  research.find_image:
    agent: cosmic/research-agent:1.0.0
    priority: normal
    fallback: null
  research.recall_session:
    agent: cosmic/research-agent:1.0.0
    priority: normal
    fallback: null
  docs.edit:
    agent: cosmic/docs-agent:2.1.0
    priority: high
    fallback: null
  docs.insert_image:
    agent: cosmic/docs-agent:2.1.0
    priority: high
    fallback: null
  docs.recall_session:
    agent: cosmic/docs-agent:2.1.0
    priority: normal
    fallback: null
```

Orchestrator dispatch algorithm:

1. Read intent from TaskEnvelope
2. Look up routing.yaml → get target agent_id
3. Query registry: is agent healthy? current_load < max_concurrency ?
4. If healthy → dispatch TaskEnvelope to agent's Redis stream at correct priority tier
5. If unhealthy → retry after backoff, escalate to DLQ after N attempts

11. Skills Registry: Storage & Lookup

The registry is two systems working together. SQLite knows what capabilities exist. Redis knows who is alive right now. The orchestrator needs both to make a good dispatch decision.

Store	Scope	Survives Restart	Updated By
SQLite	Persistent capabilities (WHAT agents can do)	Yes	Agent registration at startup
Redis	Ephemeral liveness (WHO is alive + load)	No	Agent heartbeat every N seconds

11.1 SQLite Registry Schema

```
-- registry/schema.sql
CREATE TABLE agents (
  agent_id TEXT PRIMARY KEY,
  display_name TEXT,
  status TEXT,                -- 'registered', 'deprecated'
  max_concurrency INTEGER,
  heartbeat_ttl INTEGER,
  max_task_duration_sec INTEGER, -- for XAUTOCLAIM tuning
  card_json TEXT,
  registered_at TIMESTAMP,
  updated_at TIMESTAMP
);

CREATE TABLE agent_intents (
  agent_id TEXT,
  intent TEXT,
  timeout_sec INTEGER,
  PRIMARY KEY (agent_id, intent),
  FOREIGN KEY (agent_id) REFERENCES agents(agent_id)
);
```

11.2 Redis Live State (Updated by Heartbeats)

Each agent instance writes its liveness state to Redis every N seconds. The `heartbeat_ttl` is stored in the hash itself so it is always available in the routing hot path.

```

async def heartbeat_loop(self):
    instance_key = f'registry:{self.agent_id}:{self.instance_id}'
    while True:
        await redis.hset(instance_key, mapping={
            'status': 'healthy',
            'current_load': str(self.active_task_count),
            'max_conc': str(self.max_concurrency),
            'heartbeat_ttl': str(self.heartbeat_ttl_sec),
            'last_seen': utcnow().isoformat(),
        })
        # Auto-expire the key if this instance stops heartbeating.
        # Grace period of +5s prevents flapping during GC pauses or
        # transient network blips. If the instance crashes and never
        # writes again, Redis deletes the key automatically – no
        # accumulation of dead instance keys.
        await redis.expire(instance_key, self.heartbeat_ttl_sec + 5)

        await asyncio.sleep(self.heartbeat_interval_sec)

```

11.3 Redis Intent Index (Maintained at Registration)

A Redis Set per intent, maintained at registration/deregistration time — not at query time.

```

async def register_intent_index(agent_id: str, card: dict, redis):
    for intent in card['intents']:
        intent_name = intent['name']
        await redis.sadd(f'intent:{intent_name}', agent_id)

async def deregister_intent_index(agent_id: str, card: dict, redis):
    for intent in card['intents']:
        await redis.srem(f'intent:{intent["name"]}', agent_id)

```

11.4 Orchestrator Lookup: Hot-Path Query

Intent set gives agent candidates ($O(1)$ set read), then `SCAN` finds instance keys. Never use `redis.keys()` — it blocks the entire Redis event loop.

Note: All Redis values are strings because the client uses `decode_responses=True` (see §7.2). No `.decode()` calls needed.


```

# shared/registry.py
async def find_available_instance(intent: str, redis) -> tuple[str, str] | None:
    # Step 1: SMEMBERS – O(n_agents_with_intent), typically 1-3
    agent_ids = await redis.smembers(f'intent:{intent}')

    for agent_id in agent_ids:
        # Step 2: SCAN for instance keys
        cursor = 0
        while True:
            cursor, keys = await redis.scan(
                cursor,
                match=f'registry:{agent_id}:*',
                count=10
            )
            for key in keys:
                state = await redis.hgetall(key)
                if not state:
                    continue
                load = int(state.get('current_load', '999'))
                max_c = int(state.get('max_conc', '1'))
                ttl = int(state.get('heartbeat_ttl', '30'))
                last = to_utc_aware(datetime.fromisoformat(state['last_seen']))
                is_alive = (utcnow() - last).total_seconds() < ttl
                if is_alive and load < max_c:
                    instance_id = key.split(':')[ -1]
                    return agent_id, instance_id
            if cursor == 0:
                break

    return None, None

```

11.5 Agent Startup: Self-Registration + Atomic Liveness

On startup every agent reads its own `agent_card.yaml` and writes itself into both stores. The startup sequence guarantees the agent is fully ready before receiving its first task.

```

async def register(self):
    card = yaml.safe_load(open('agent_card.yaml'))

    # Step 1: Write SQLite (capability declaration)
    db.execute('''
        INSERT OR REPLACE INTO agents
        (agent_id, display_name, max_concurrency, heartbeat_ttl,
         max_task_duration_sec, card_json, registered_at)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', [card['agent_id'], card['display_name'],
          card['sla']['max_concurrency'],
          card['sla']['heartbeat_ttl_sec'],
          card['sla']['max_task_duration_sec'],
          json.dumps(card), utcnow()])

    for intent in card['intents']:
        db.execute('''
            INSERT OR REPLACE INTO agent_intents
            (agent_id, intent, timeout_sec)
            VALUES (?, ?, ?)
        ''', [card['agent_id'], intent['name'], intent['timeout_sec']])

    # Step 2: Register intent index in Redis
    await register_intent_index(card['agent_id'], card, redis)

    # Step 3: Write initial Redis liveness entry BEFORE consuming tasks
    instance_key = f'registry:{self.agent_id}:{self.instance_id}'
    await redis.hset(instance_key, mapping={
        'status': 'starting',
        'current_load': '0',
        'max_conc': str(self.max_concurrency),
        'heartbeat_ttl': str(self.heartbeat_ttl_sec),
        'last_seen': utcnow().isoformat(),
    })

    # Step 4: Run any startup checks (DB migrations, model load, etc)
    await self.on_startup()

    # Step 5: Mark healthy – orchestrator will now dispatch to this instance
    await redis.hset(instance_key, 'status', 'healthy')

    # Step 6: Start heartbeat loop
    asyncio.create_task(self.heartbeat_loop())

```

```
# Step 7: NOW start consuming from Redis streams
await self.run()
```

Startup sequence contract:

SQLite write → Redis 'starting' → on_startup() → Redis 'healthy' → begin consuming. An agent stuck in starting beyond startup_timeout_sec is marked dead. Orchestrator routing requires status == 'healthy'.

12. Agent Learnings, Session Data & Input Artifacts

12.1 Agent Learnings (store/learnings.md)

Each agent accumulates knowledge over time in store/learnings.md — facts, preferences, and patterns it discovers across sessions. This is the agent's long-term memory.

Research Agent – Learnings

User Preferences

- User prefers academic sources over blog posts
- User wants APA citation format

Source Quality Notes

- arxiv.org: high reliability for CS topics
- medium.com: treat as opinion, always cross-reference

Session Patterns

- When intent is research.topic with input containing "architecture", user typically wants both academic papers and industry blog posts

The agent reads learnings.md at task start (alongside the system prompt) and appends to it after completing tasks when new knowledge is worth persisting. This file is agent-private — no other agent reads it directly. However, the Session Manager syncs agent learnings into the memory/agent_notes/ directory and indexes them in Qdrant as high-priority memories (see §23.7). This allows the orchestrator and all LLM backends to benefit from agent-curated knowledge when assembling context.

12.2 Agent-Managed Session Data (store/data/)

Each agent manages its own session storage in store/data/ with whatever schema fits its domain. **No uniform session schema is imposed across agents.** A research agent might track source credibility; a

docs agent might track edit history and document versions.

Example: docs agent's schema:

```
-- agents/docs_agent/store/data/sessions.db
CREATE TABLE edit_sessions (
  session_id TEXT,
  task_id TEXT,
  doc_id TEXT,
  edit_type TEXT,          -- 'insert', 'delete', 'reformat'
  section TEXT,
  summary TEXT,
  before_hash TEXT,
  after_hash TEXT,
  created_at TIMESTAMP,
  PRIMARY KEY (session_id, task_id)
);
```

Example: research agent's schema:

```
-- agents/research_agent/store/data/sessions.db
CREATE TABLE research_sessions (
  session_id TEXT,
  task_id TEXT,
  query TEXT,
  sources_json TEXT,
  citations_count INTEGER,
  confidence_score REAL,
  created_at TIMESTAMP,
  PRIMARY KEY (session_id, task_id)
);
```

12.3 Session Context Flow

The orchestrator maintains session-wide context and passes relevant information to agents via the TaskEnvelope input field. Agents do **not** read from each other's databases or from a shared session store.

Orchestrator maintains session context

- |
- | — Passes relevant context in TaskEnvelope.input
 - | e.g., input: { query: "...", session_context: { doc_id: "...", goal: "..." } }
- |
- | — Queries agent about past work via recall intents
 - | e.g., intent: "docs.recall_session", input: { session_id: "sess_abc" }
 - | Agent queries its own store/data/ and returns structured history
- |
- | — Promotes important facts to future task inputs
 - | e.g., research_agent discovered "user prefers academic sources"
 - | → orchestrator includes this in future tasks to other agents

Why no shared session database: Agents run as separate processes (potentially separate containers). A shared SQLite file requires filesystem-level coordination. A shared Redis store couples all agents' session schemas. Instead, the orchestrator asks agents about their past work using recall intents, and passes relevant context forward — keeping agents fully decoupled.

12.4 Recall Intents: Querying Agents About Past Work

When the orchestrator needs an agent's session history (e.g., user asks "explain the last edits we did"), it dispatches a recall intent. The agent queries its own store/data/ and returns structured results.

```
# Orchestrator dispatches to docs_agent:
```

```
TaskEnvelope(  
    task_id='tsk_recall_001',  
    session_id='sess_abc',  
    sender='cosmic/orchestrator:1.0.0',  
    recipient='cosmic/docs-agent:2.1.0',  
    intent='docs.recall_session',  
    input={  
        'session_id': 'sess_abc',  
        'query': 'what edits were made',  
        'limit': 10,  
    },  
    ...  
)
```

```
# docs_agent handles this by querying its own sessions.db:
```

```
# SELECT * FROM edit_sessions WHERE session_id = 'sess_abc' ORDER BY created_at DESC LIMIT 10
```

```
# Returns structured AgentResult with edit history in output dict
```

Each agent defines its own recall intent schema in `schemas/intents/`. The orchestrator doesn't need to know the agent's internal data model — it just sends the query and receives structured output.

12.5 Input Artifacts: Passing Files TO Agents

`input_artifacts` allows the orchestrator to pass files — PDFs, images, prior results — as typed, validated inputs to any agent.

```
task = TaskEnvelope(  
    task_id='tsk_analyze_001',  
    session_id='sess_abc',  
    sender='cosmic/orchestrator:1.0.0',  
    recipient='cosmic/research-agent:1.0.0',  
    intent='research.topic',  
    input={'query': 'summarize key architecture decisions'},  
    input_artifacts=[  
        ArtifactManifest(  
            artifact_id='art_001',  
            task_id='tsk_prev_001',  
            mime='application/pdf',  
            sha256='abc123...',  
            path='runs/artifacts/tsk_prev_001/paper.pdf',  
            created_by_agent='cosmic/docs-agent:2.1.0'  
        )  
    ]  
)
```

Why `input_artifacts` is not just a path string in `input: dict`: Typed (agent knows mime type), verified (sha256 integrity check), traceable (links back to producing task), reusable (same artifact can be passed to multiple agents without copying), auditable (orchestrator knows exactly which files were passed to which agents).

12.6 Agent Task Handler

Every agent base class handles session data loading, artifact verification, and idempotency.

```

async def handle(self, task: TaskEnvelope, msg_id: str, stream: str):
    # Verify signature
    if not verify_incoming(task):
        raise AuthError(f'Invalid signature on task {task.task_id}')

    # Check epoch before any execution
    if task.leader_epoch is not None:
        current_epoch = int(await redis.get(EPOCH_KEY) or 0)
        if task.leader_epoch < current_epoch:
            await self._reject_stale_epoch(task, msg_id, stream, current_epoch)
            return

    # — Extract and isolate auth context —————
    # input.auth is stripped from the task input BEFORE any processing.
    # It is exposed to the agent's execute() method via self.auth —
    # a runtime-only attribute that is NEVER serialized to events,
    # logs, artifacts, store/, or learnings.md.
    self.auth = task.input.pop('auth', None)

    # Verify input artifacts
    for artifact in task.input_artifacts:
        verify_artifact(artifact)

    # Execute with idempotency enforcement
    result = await execute_with_idempotency(
        task, self.execute, redis,
        agent_max_duration_sec=self.max_task_duration_sec,
    )

    # Clear auth context after execution — never persisted
    self.auth = None

    if isinstance(result, AgentResult):
        # Update agent's own session data + learnings as needed
        if task.session_id and result.status == 'completed':
            self.save_session_data(task.session_id, task, result)
            self.maybe_update_learnings(task, result)
        await redis.xack(stream, 'workers', msg_id)
        await self.emit_terminal_event(task.task_id, result)

    elif isinstance(result, TaskInProgress):
        # Another instance is executing. Recovery is orchestrator-owned.
        #
        # ORDER IS CRITICAL: emit task.deferred BEFORE XACK.
        # If agent crashes after XACK but before emit, the message is gone

```

```

# and no deferred_checks row is created – task is silently lost.
# By emitting first, the orchestrator has the deferred record in its
# event stream before we remove the message from pending.
# If agent crashes after emit but before XACK: message is reclaimed
# by XAUTOCLAIM → idempotency returns TaskInProgress again → another
# task.deferred is emitted (harmless duplicate, orchestrator upserts).
await self.emit_event(
    task_id=task.task_id,
    event_type='task.deferred',
    payload={
        'reason': 'already_executing',
        'idempotency_key': task.idempotency_key,
        'executing_since': result.executing_since.isoformat(),
        'check_after_sec': result.check_after_sec,
    },
)
await redis.xack(stream, 'workers', msg_id)

```

12.7 Deferred Task Recovery (Orchestrator-Owned)

When an agent emits `task.deferred`, the orchestrator — not the agent — owns the recovery lifecycle. This preserves the "all communication is orchestrator-mediated" rule and ensures recovery survives process crashes.

Why the orchestrator, not the agent?

- Agent-side timers (`asyncio.sleep`) block the worker loop and are not durable — a crash loses the timer.
- Agent-side redispatch bypasses the leader ledger and fencing controls.
- The orchestrator already consumes `streams:events` and already has crash recovery for orphaned tasks.

Durable state: The orchestrator persists deferred tasks in its own SQLite ledger so recovery survives crashes.


```
-- Orchestrator's task ledger (already exists for task tracking)
CREATE TABLE deferred_checks (
    task_id TEXT PRIMARY KEY,
    idempotency_key TEXT NOT NULL,
    check_after TIMESTAMP NOT NULL,      -- UTC: when to next check
    attempts INTEGER DEFAULT 0,
    max_attempts INTEGER DEFAULT 6,
    created_at TIMESTAMP NOT NULL
);
```

Orchestrator event handler:

```
async def on_event(event: EventEnvelope):
    if event.event_type == 'task.deferred':
        # Persist durable check – survives orchestrator crash
        db.execute('''
            INSERT OR REPLACE INTO deferred_checks
            (task_id, idempotency_key, check_after, attempts, max_attempts, created_at)
            VALUES (?, ?, datetime('now', '+' || ? || ' seconds'), 0, 6, datetime('now'))
        ''', [
            event.task_id,
            event.payload['idempotency_key'],
            event.payload['check_after_sec'],
        ])
```

Orchestrator polling loop (runs alongside the main event consumer):

```

async def deferred_check_loop(redis):
    """Non-blocking poll – runs every 10s, never sleeps for minutes."""
    while True:
        await asyncio.sleep(10)      # poll interval – not per-task blocking

        due = db.execute('''
            SELECT task_id, idempotency_key, attempts, max_attempts
            FROM deferred_checks
            WHERE check_after <= datetime('now')
        ''').fetchall()

        for task_id, idem_key, attempts, max_attempts in due:
            result_key = f'idempotency:result:{idem_key}'
            dedupe_key = f'idempotency:{idem_key}'

            # Case 1: result exists – task completed, clean up
            if await redis.exists(result_key):
                db.execute('DELETE FROM deferred_checks WHERE task_id = ?', [task_id])
                continue

            # Case 2: lock still held – executor likely still running.
            # Keep checking regardless of attempt count while lock exists.
            # The lock has its own TTL – it will expire if executor is dead.
            if await redis.exists(dedupe_key):
                db.execute('''
                    UPDATE deferred_checks
                    SET check_after = datetime('now', '+30 seconds'),
                        attempts = attempts + 1
                    WHERE task_id = ?
                ''', [task_id])
                # Alert after max_attempts so operators know, but don't redrive
                if attempts >= max_attempts:
                    logger.warning(
                        f'Deferred check for {task_id}: {attempts} attempts, '
                        f'lock still held – executor may be stuck'
                    )
                continue

            # Case 3: lock gone – executor finished without storing result,
            # or crashed and lock expired. Safe to redrive.
            logger.warning(f'Deferred recovery: redriving {task_id}')
            task = await load_task_envelope(task_id)
            new_epoch = await get_current_epoch(redis)
            task = prepare_for_redispatch(task, new_epoch, self.secrets)

```

```
await dispatch(task, redis)
db.execute('DELETE FROM deferred_checks WHERE task_id = ?', [task_id])
```

Crash safety: On orchestrator startup, `recover_orphaned_tasks()` (§16.3) already scans for stale in-progress tasks. `deferred_checks` rows survive in SQLite. The polling loop picks them up immediately after restart. Worst-case recovery time: poll interval (10s) + one check cycle (30s) = 40s after orchestrator restarts.

13. Bidirectional Communication: Agent ↔ Orchestrator

Core rule: All communication is orchestrator-mediated. Agents NEVER talk to each other directly. All messages route through the orchestrator. This preserves the task ledger, observability, and retry logic for every interaction.

13.1 Case 1: Agent Asks Orchestrator (Reverse Task)

Agent hits ambiguity it cannot resolve. It creates a `TaskEnvelope` addressed TO the orchestrator, suspends its current task, and waits for a reply.

```

# Agent sends a clarification request
{
  'task_id': 'tsk_question_001',
  'parent_task_id': 'tsk_abc123',
  'sender': 'cosmic/research-agent:1.0.0',
  'recipient': 'cosmic/orchestrator:1.0.0',
  'intent': 'orchestrator.clarify',
  'input': {
    'question': 'Found conflicting sources. Which do I prioritize?',
    'options': ['source_a', 'source_b'],
    'context_artifact': 'art_xyz'
  },
  'idempotency_key': 'uuid'
}

# Agent emits task.suspended event for parent task
# Serializes in-progress state to runtime/state.db
# Waits on streams:cosmic/research-agent:1.0.0:replies

# Orchestrator processes clarify intent, replies with TaskEnvelope:
# intent: 'agent.resume', input: { decision: 'use source_a', task_id: 'tsk_abc123' }
# Agent deserializes state, resumes work

```

Orchestrator must define these intents:

Intent	Purpose
orchestrator.clarify	Agent needs a decision between options
orchestrator.approve	Agent needs permission before a side effect
orchestrator.decide	Agent hit a branch it cannot resolve
orchestrator.delegate	Agent needs another agent's output
orchestrator.escalate	Push to gateway for human input
orchestrator.refresh_credential	Agent's access token expired mid-task — orchestrator refreshes via Gateway and resumes agent with new token (see §22.5)

13.2 Case 2: Agent Needs Human Input

Orchestrator cannot answer either. It escalates to the gateway which surfaces it to the user via WebSocket.

```

# Full escalation chain:
# 1. research_agent → orchestrator (intent: orchestrator.clarify)
# 2. Orchestrator cannot resolve → escalates to gateway
#    intent: user.input_required
#    input: { question, options, blocking_task_id }
# 3. Gateway surfaces to user via WebSocket (type: 'clarify')
# 4. User answers → WebSocket message (type: 'reply')
# 5. Gateway → orchestrator → research_agent.resume
#
# Task state throughout: task.suspended
# Timeout: if user doesn't answer within N seconds → task.failed
#    with error.code = 'HUMAN_INPUT_TIMEOUT'

```

13.3 Case 3: Agent Needs Another Agent

Route through orchestrator. Never direct agent-to-agent.

```

# WRONG – do not do this:
# result = await docs_agent.get_context(doc_id) # bypasses task ledger

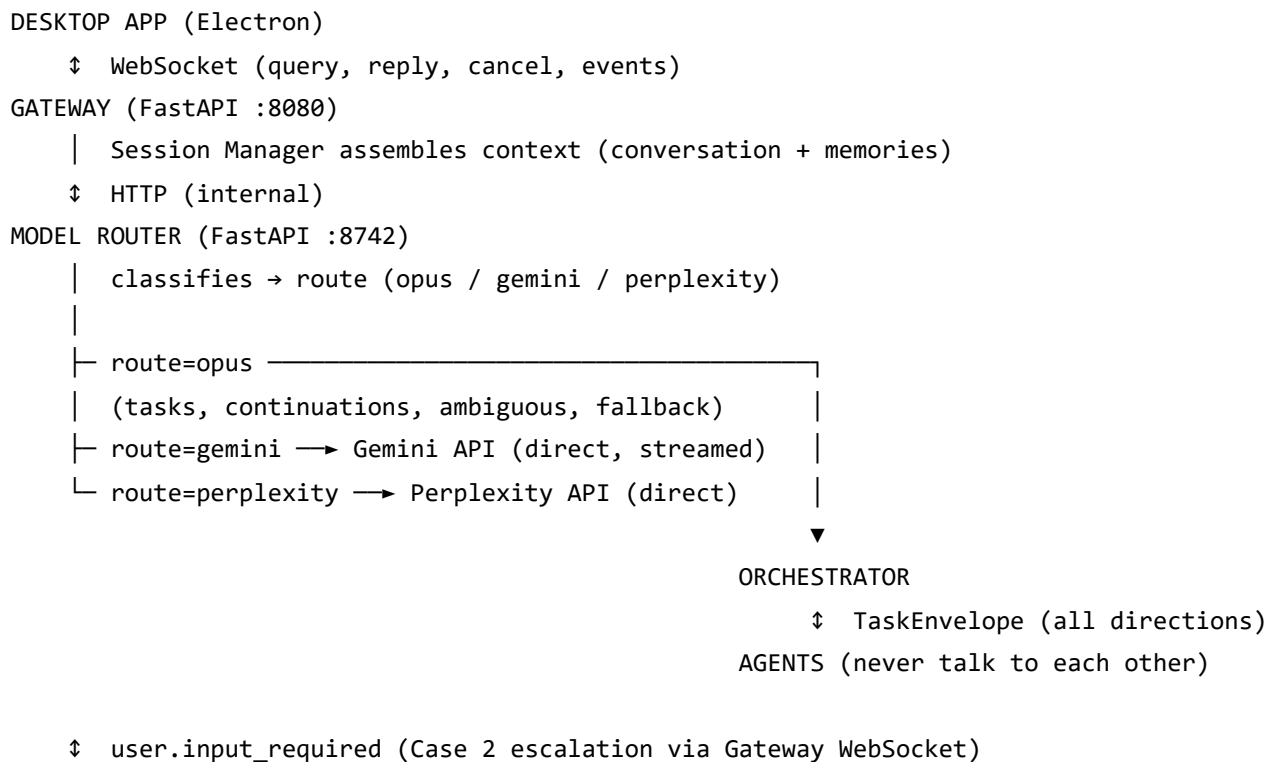
# CORRECT – send reverse task to orchestrator:
{
  'task_id': 'tsk_ctx_001',
  'parent_task_id': 'tsk_abc123',
  'sender': 'cosmic/research-agent:1.0.0',
  'recipient': 'cosmic/orchestrator:1.0.0',
  'intent': 'orchestrator.delegate',
  'input': {
    'target_intent': 'docs.get_section_context',
    'input': { 'doc_id': 'doc_xyz', 'section': 'architecture' }
  }
}
# Orchestrator fans out to docs_agent, waits for result,
# then returns result to research_agent via resume envelope.

```

13.4 Task Lifecycle (Suspendable Tasks)

```
task.accepted
  ↓
task.progress (emitted as work proceeds)
  ↓
task.suspended ← agent waiting for input before continuing
  ↓ (agent serializes state to runtime/state.db)
... question resolved, resume envelope arrives ...
  ↓
task.resumed → task.progress (continues)
  ↓
task.completed OR task.failed OR task.suspended again
```

13.5 Communication Flow Diagram



All orchestrator-mediated messages are TaskEnvelopes.

The orchestrator is both a dispatcher AND a participant in the task graph.

All three routes receive the same assembled context from the Session Manager.

14. Idempotency Enforcement

Every task execution is guarded by a SETNX-based idempotency check. The ordering of operations is critical and non-negotiable.

14.1 Configuration

```
# shared/config.py
DEDUPE_TTL_MIN_SEC = 60
DEDUPE_TTL_FALLBACK_SEC = 600           # only if registry lookup fails
RESULT_TTL_SEC = 604800                 # 7 days
```

TTL Key	Purpose
dedupe_key TTL	Prevents re-execution of in-flight or completed task. Derived from <code>2 × deadline</code> if deadline exists, otherwise <code>2 × max_task_duration_sec</code> from the agent's registry entry. Floor: 60s.
result_key TTL	How long terminal results are cached for replay and auditing. Not safety-critical. Default: 7 days.

14.2 Canonical `execute_with_idempotency`

Order: (1) replay check, (2) deadline guard, (3) lock + execute. This ordering is not optional.

```

# shared/idempotency.py
async def execute_with_idempotency(
    task: TaskEnvelope,
    handler: Callable,
    redis: Redis,
    agent_max_duration_sec: int = 0,    # from agent_card.yaml sla.max_task_duration_sec
) -> ExecutionResult:

    dedupe_key = f'idempotency:{task.idempotency_key}'
    result_key = f'idempotency:result:{task.idempotency_key}'

    # — Step 1: Replay path – ALWAYS first —————
    stored = await redis.get(result_key)
    if stored:
        return AgentResult.model_validate_json(stored)

    # — Step 2: Deadline guard – only for tasks not yet executed —————
    if task.deadline_ts is not None:
        remaining_sec = deadline_remaining_sec(task.deadline_ts)
        if remaining_sec <= 0:
            return AgentResult(
                status='failed',
                output={},
                artifacts=[],
                error=AgentError(
                    code='DEADLINE_EXCEEDED',
                    retryable=False,
                    message='Task deadline already exceeded before execution',
                    next_action='skip',
                ),
            )
        base_ttl = int(remaining_sec) * 2
    elif agent_max_duration_sec > 0:
        # No deadline – derive from agent's declared max task duration
        base_ttl = agent_max_duration_sec * 2
    else:
        base_ttl = DEDUPE_TTL_FALLBACK_SEC

    dedupe_ttl = max(DEDUPE_TTL_MIN_SEC, base_ttl)

    # — Step 3: Acquire lock —————
    exec_since = utcnow().isoformat()
    acquired = await redis.set(dedupe_key, exec_since, nx=True, ex=dedupe_ttl)

    if not acquired:

```



```

raw = await redis.get(dedupe_key)
since_str = raw if raw else exec_since
return TaskInProgress(
    task_id=task.task_id,
    idempotency_key=task.idempotency_key,
    executing_since=datetime.fromisoformat(since_str),
    check_after_sec=30,
)

```

— Step 4: Execute _____

```

try:
    result = await handler(task)
    await redis.set(result_key, result.model_dump_json(), ex=RESULT_TTL_SEC)
    return result
except Exception:
    await redis.delete(dedupe_key)
    raise

```

Critical invariant: If a terminal result is stored → replay it, always, regardless of deadline. If no result stored AND deadline exceeded → DEADLINE_EXCEEDED . If no result stored AND deadline not exceeded → acquire lock and execute.

Store-before-ack ordering: result stored → stream message acked → terminal event emitted . If crash happens between store and ack: message redelivered, result replayed (safe). If crash happens between ack and event: result is stored, orchestrator can query it (safe).

15. Security

15.1 HMAC Signing: Full Payload + Per-Service Secrets

Every TaskEnvelope is signed with the recipient's secret. A compromised agent can only forge messages to itself, not to other agents.

```

# shared/auth.py
import hmac, hashlib, json, os

def canonical_payload(task: TaskEnvelope) -> bytes:
    data = task.model_dump(
        mode='json',
        exclude={'signature'},
    )
    return json.dumps(
        data,
        sort_keys=True,
        separators=(',', ':'),
    ).encode('utf-8')

def sign_task(task: TaskEnvelope, secret: str) -> str:
    return hmac.new(
        secret.encode(),
        canonical_payload(task),
        hashlib.sha256
    ).hexdigest()

def verify_task(task: TaskEnvelope, signature: str, secret: str) -> bool:
    return hmac.compare_digest(sign_task(task, secret), signature)

def dispatch_signed(task: TaskEnvelope, secrets: dict) -> TaskEnvelope:
    secret = secrets[task.recipient]
    task.signature = sign_task(task, secret)
    return task

def verify_incoming(task: TaskEnvelope) -> bool:
    secret = os.environ['AGENT_SECRET']
    return verify_task(task, task.signature, secret)

```

Per-service secret model: Each agent has its OWN secret set via supervisord environment variable. The orchestrator holds all secrets. Blast radius: compromising `research_agent` 's secret only allows forging tasks TO `research_agent` .

15.2 Artifact Path Security

Artifact paths are validated against an allowlist before any file read. Path traversal attacks are blocked before sha256 verification.

```

# shared/artifact_security.py
from pathlib import Path
import hashlib

PROJECT_ROOT = Path(__file__).resolve().parent.parent

STATIC_ROOTS = [
    (PROJECT_ROOT / 'runs' / 'artifacts').resolve(),
]

def _get_allowed_roots() -> list[Path]:
    """Build allowlist dynamically on every call.
    Includes all agents/*/store/ and agents/*/runtime/ directories,
    even those created after process startup."""
    roots = list(STATIC_ROOTS)
    agents_dir = PROJECT_ROOT / 'agents'
    if agents_dir.exists():
        for agent_dir in agents_dir.iterdir():
            if not agent_dir.is_dir():
                continue
            for subdir in ['store', 'runtime']:
                path = agent_dir / subdir
                if path.exists():
                    roots.append(path.resolve())
    return roots

def safe_artifact_path(raw_path: str) -> Path:
    # Resolve relative paths against PROJECT_ROOT, not process CWD.
    # This prevents environment-dependent behavior when agents run
    # from different working directories.
    candidate = Path(raw_path)
    if not candidate.is_absolute():
        candidate = PROJECT_ROOT / candidate
    resolved = candidate.resolve()
    allowed = any(resolved.is_relative_to(root) for root in _get_allowed_roots())
    if not allowed:
        raise SecurityError(f'Path outside allowlist: {resolved}')
    return resolved

def verify_artifact(manifest: ArtifactManifest) -> Path:
    safe_path = safe_artifact_path(manifest.path)
    actual_hash = hashlib.sha256(safe_path.read_bytes()).hexdigest()
    if actual_hash != manifest.sha256:
        raise IntegrityError(
            f'sha256 mismatch for {manifest.artifact_id}: '

```

```

        f'expected {manifest.sha256}, got {actual_hash}'
    )
    return safe_path

```

15.3 Secret Management

All API keys and secrets **must** be externalized to environment variables. Never hardcode secrets in source files.

Secret	Owner	Environment Variable
Groq API key	Model Router	GROQ_API_KEY
Gemini API key	Gateway (Gemini adapter)	GEMINI_API_KEY
Perplexity API key	Gateway (Perplexity adapter)	PERPLEXITY_API_KEY
Gateway local token	Gateway + Desktop App	GATEWAY_LOCAL_TOKEN
Per-agent HMAC secrets	Orchestrator (all), Agents (own)	AGENT_SECRET / AGENT_SECRETS
Gateway internal service token	Gateway + Orchestrator	GATEWAY_INTERNAL_TOKEN
Credential encryption key	Gateway (Credential Manager)	CREDENTIAL_ENCRYPTION_KEY
OAuth client secrets (per provider)	Gateway (Credential Manager)	OAuth_GOOGLE_CLIENT_SECRET , OAuth_GITHUB_CLIENT_SECRET , etc.
OpenRouter API key	Gateway (Session Manager — embeddings)	OPENROUTER_API_KEY

Storage: In development, secrets live in `.env` files (gitignored). In production, use a secrets manager or encrypted environment injection (Docker secrets, Kubernetes secrets, systemd `CredentialDirectory`).

Rotation: HMAC agent secrets can be rotated by updating the orchestrator's `AGENT_SECRETS` map and each agent's `AGENT_SECRET` in a rolling deploy. The contract version mechanism (§21) ensures in-flight envelopes signed with the old key are drained before the old key is removed.

Credential encryption key rotation: Generate a new Fernet key, re-encrypt all stored refresh tokens under the new key (batch migration), then swap the env var. During migration, the Credential Manager should accept both old and new keys (try new first, fall back to old). See §22 for details.

16. Leader Election + Fencing

The orchestrator uses leader election with monotonic epoch/fencing tokens to prevent split-brain writes.

16.1 Leader Election via Redis

```
# shared/leader.py
LEADER_KEY = 'orchestrator:leader'
EPOCH_KEY = 'orchestrator:epoch'
LEADER_TTL = 15
RENEW_EVERY = 5

ACQUIRE_SCRIPT = '''
local ok = redis.call('set', KEYS[1], ARGV[1], 'NX', 'EX', ARGV[2])
if ok then
    local epoch = redis.call('incr', KEYS[2])
    return epoch
else
    return 0
end
'''

async def acquire_leadership(instance_id: str, redis) -> int | None:
    result = await redis.eval(
        ACQUIRE_SCRIPT, 2,
        LEADER_KEY, EPOCH_KEY,
        instance_id, str(LEADER_TTL)
    )
    return int(result) if result else None

RENEW_SCRIPT = '''
if redis.call('get', KEYS[1]) == ARGV[1]
    and tostring(redis.call('get', KEYS[2])) == ARGV[2] then
    return redis.call('expire', KEYS[1], ARGV[3])
else return 0 end
'''

async def renew_leadership(instance_id: str, epoch: int, redis) -> bool:
    result = await redis.eval(
        RENEW_SCRIPT, 2,
        LEADER_KEY, EPOCH_KEY,
        instance_id, str(epoch), str(LEADER_TTL)
    )
    return bool(result)
```

16.2 Orchestrator Main Loop

```
async def orchestrator_main(instance_id: str, redis):
    while True:
        epoch = await acquire_leadership(instance_id, redis)
        if epoch:
            logger.info(f'{instance_id} elected leader, epoch={epoch}')
            asyncio.create_task(renew_loop(instance_id, epoch, redis))
            await recover_orphaned_tasks(redis, epoch, self.secrets)
            await run_as_leader(epoch)
        else:
            await asyncio.sleep(LEADER_TTL)

async def renew_loop(instance_id: str, epoch: int, redis):
    while True:
        await asyncio.sleep(RENEW_EVERY)
        still_leader = await renew_leadership(instance_id, epoch, redis)
        if not still_leader:
            logger.error(f'Lost leadership (epoch={epoch}), halting dispatch')
            raise LeadershipLost()
```

16.3 Crash Recovery

```
async def recover_orphaned_tasks(redis, epoch, secrets):
    orphaned = db.execute('''
        SELECT task_id, recipient, priority FROM tasks
        WHERE status IN ('accepted', 'in_progress', 'suspended')
        AND updated_at < datetime('now', '-60 seconds')
    ''').fetchall()
    for task_id, recipient, priority in orphaned:
        logger.warning(f'Recovering orphaned task: {task_id}')
        task = load_task_envelope(task_id)
        task = prepare_for_redispatch(task, epoch, secrets)
        await dispatch(task, redis)
```

16.4 Stale Epoch Rejection + Redrive

When an agent receives a task with a stale `leader_epoch`, it acks the message (removes from pending), does NOT execute, and emits `task.rejected`. The new leader redispatches with the current epoch.

```

# Agent-side
async def _reject_stale_epoch(self, task, msg_id, stream, current_epoch):
    await redis.xack(stream, 'workers', msg_id)
    await self.emit_event(
        task_id=task.task_id,
        event_type='task.rejected',
        payload={
            'reason': 'stale_epoch',
            'task_epoch': task.leader_epoch,
            'current_epoch': current_epoch,
        },
    )

# Orchestrator-side
async def on_event(event: EventEnvelope):
    if event.event_type == 'task.rejected':
        if event.payload.get('reason') == 'stale_epoch':
            task = await load_task_envelope(event.task_id)
            new_epoch = await get_current_epoch(redis)
            task = prepare_for_redispatch(task, new_epoch, self.secrets)
            # idempotency_key is UNCHANGED – stale reject = no execution happened
            await dispatch(task, redis)

```

17. Atomic Seq Allocation

seq on EventEnvelopes must be unique and monotonically increasing per task. With multiple worker instances, allocation must be atomic.


```

# shared/events.py
SEQ_KEY_TTL_SEC = RESULT_TTL_SEC

async def next_seq(task_id: str, redis) -> int:
    return int(await redis.incr(f'event_seq:{task_id}'))

async def expire_seq_key(task_id: str, redis):
    await redis.expire(f'event_seq:{task_id}', SEQ_KEY_TTL_SEC)

async def emit_event(self, task_id: str, event_type: str, payload: dict):
    seq = await next_seq(task_id, self.redis)
    event = EventEnvelope(
        task_id=task_id,
        agent_id=self.agent_id,
        event_type=event_type,
        seq=seq,
        payload=payload,
        emitted_at=utcnow(),
    )
    await self.redis.xadd('streams:events', {'event': event.model_dump_json()})

    if event_type in TERMINAL_EVENTS:
        await expire_seq_key(task_id, self.redis)

```

Never use local counters or timestamps as seq — they are not safe across instances.

18. Priority Fairness: Aging-Based Promotion

Strict high → normal → low polling starves low-priority tasks under sustained load. Aging promotes waiting tasks.

18.1 Aging Thresholds

```

AGING_THRESHOLDS = {
    'low': 120,          # promote to normal after 2 min
    'normal': 60,        # promote to high after 1 min
}

```

18.2 Worker Loop Integration

The worker loop calls `_priority_order_with_aging()` before each consumption cycle. This method inspects `XPENDING` summaries ($O(1)$ per call) to detect aged messages and reorders the consumption priority.

```
async def _priority_order_with_aging(self) -> list[str]:
    """Check if lower-priority streams have messages that exceed aging
    thresholds. If so, consume from the aged stream first.

    Uses XPENDING summary form —  $O(1)$ , returns min/max message IDs
    and pending count. Cost: 2 Redis calls per cycle (one per lower
    priority tier). Acceptable overhead for fairness guarantee."""
    for original, threshold in [
        ('low', AGING_THRESHOLDS['low']),
        ('normal', AGING_THRESHOLDS['normal']),
    ]:
        info = await self.redis.xpending(
            STREAMS[original], 'workers',
        )
        # info = {'pending': N, 'min': '<id>', 'max': '<id>', 'consumers': [...]}
        if info and info['pending'] > 0 and info['min']:
            oldest_ts_ms = int(info['min'].split('-')[0])
            age_sec = (time.time() * 1000 - oldest_ts_ms) / 1000
            if age_sec > threshold:
                # Aged message found — promote this stream to front of queue
                order = [original] + [
                    p for p in ['high', 'normal', 'low'] if p != original
                ]
                return order

    return ['high', 'normal', 'low']    # default strict ordering
```

How it works with the worker loop (§8.4): The worker loop calls

`consume_order = await self._priority_order_with_aging()` and then iterates streams in the returned order. When a low-priority message has been pending for over 120 seconds, the worker consumes from `low` first, then `high`, then `normal`. This ensures no priority tier is permanently starved while preserving priority ordering under normal conditions.

Performance impact: Two `XPENDING` summary calls per iteration ($O(1)$ each). Under typical load (sub-second iterations), this adds $< 1\text{ms}$ overhead. The aging check only changes behavior when messages have been waiting for minutes — it does not affect the fast path.

19. DLQ + Retry Policy

19.1 Retry Behavior

Retry policy is per intent, declared in `agent_card.yaml` under `sla.retry_policy`.

Error Code	Behavior
TIMEOUT , NETWORK_ERROR , RATE_LIMITED	Retryable — exponential backoff up to <code>backoff_max_sec</code> , up to <code>max_attempts</code>
INVALID_INPUT , AUTH_ERROR , SCHEMA_VIOLATION	Non-retryable — straight to DLQ
DEADLINE_EXCEEDED	Non-retryable — pre-flight rejection, never executed

19.2 DLQ Routing

After `max_attempts` : XADD `streams:dlq` with full task envelope + error history. The DLQ record includes: original `task_id` , all attempt timestamps, all error codes, final error message. The DLQ is inspected manually or by a separate remediation agent.

20. Broadcast & Dynamic Capability Discovery

20.1 Pattern A: Capability Advertisement (Push — Default)

Agents publish to a shared stream when capabilities change. Orchestrator subscribes and updates the registry live. Zero latency on dispatch.

```
await redis.xadd('streams:capability.updates', {
  'agent_id': 'cosmic/research-agent:1.0.0',
  'event': 'capability.added',
  'intent': 'research.summarize_pdf',
  'available_from': utcnow().isoformat()
})
```

20.2 Pattern B: Capability Query Broadcast (Pull — Dynamic Discovery)

Use only for genuinely unknown external agents at runtime.

```
await redis.xadd('streams:broadcast', {
  'type': 'capability.query',
  'query_id': 'qry_abc123',
  'intent_needed': 'research.summarize_pdf',
  'reply_to': 'streams:cosmic/orchestrator:1.0.0:replies',
  'deadline_ts': (utcnow() + timedelta(seconds=5)).isoformat()
})
```

Pattern	When to Use
A (push)	Your agents. Known at deploy time. 99% of cases. Zero dispatch latency.
B (broadcast)	External agents, plugins, dynamically loaded specialists. Adds ~deadline_ts latency. Always cache result back into SQLite.

21. Rolling Deploy Compatibility

Safe deployment requires readers to be upgraded before writers. Never advance writers past what the oldest live reader can parse.

21.1 Version Configuration

contract_version on envelopes is a **dynamic string** set from config, not a hardcoded literal. This allows the same codebase to write different versions during a rolling deploy.

Enforcement is at the model level. Both TaskEnvelope and EventEnvelope have a @field_validator on contract_version that rejects any version not in ACCEPTED_READ_VERSIONS at **deserialization** time. This is the inbound (read) check.

Outbound (write) validation is a separate function called at dispatch time. This decouples the rules: during a rolling deploy, readers accept {'1.6', '1.7'} while writers are constrained to emit only CURRENT_WRITE_VERSION .

```
# shared/config.py
CURRENT_WRITE_VERSION = '1.6'
ACCEPTED_READ_VERSIONS = {'1.6'}

def validate_outbound_version(envelope) -> None:
    """Called at dispatch time – ensures we only write the current version."""
    if envelope.contract_version != CURRENT_WRITE_VERSION:
        raise ContractVersionError(
            f'Outbound envelope has version {envelope.contract_version}, '
            f'but CURRENT_WRITE_VERSION is {CURRENT_WRITE_VERSION}'
        )
```

21.2 Three-Phase Deploy

Phase	Writers	Readers	Purpose
Phase 1 (deploy readers)	CURRENT_WRITE_VERSION = '1.6'	ACCEPTED_READ_VERSIONS = {'1.6', '1.7'}	All nodes can parse both versions
Phase 2 (flip writers)	CURRENT_WRITE_VERSION = '1.7'	ACCEPTED_READ_VERSIONS = {'1.6', '1.7'}	New envelopes flow, old readers already handle them
Phase 3 (narrow window)	CURRENT_WRITE_VERSION = '1.7'	ACCEPTED_READ_VERSIONS = {'1.7'}	Cluster fully on new version

Deploy checklist:

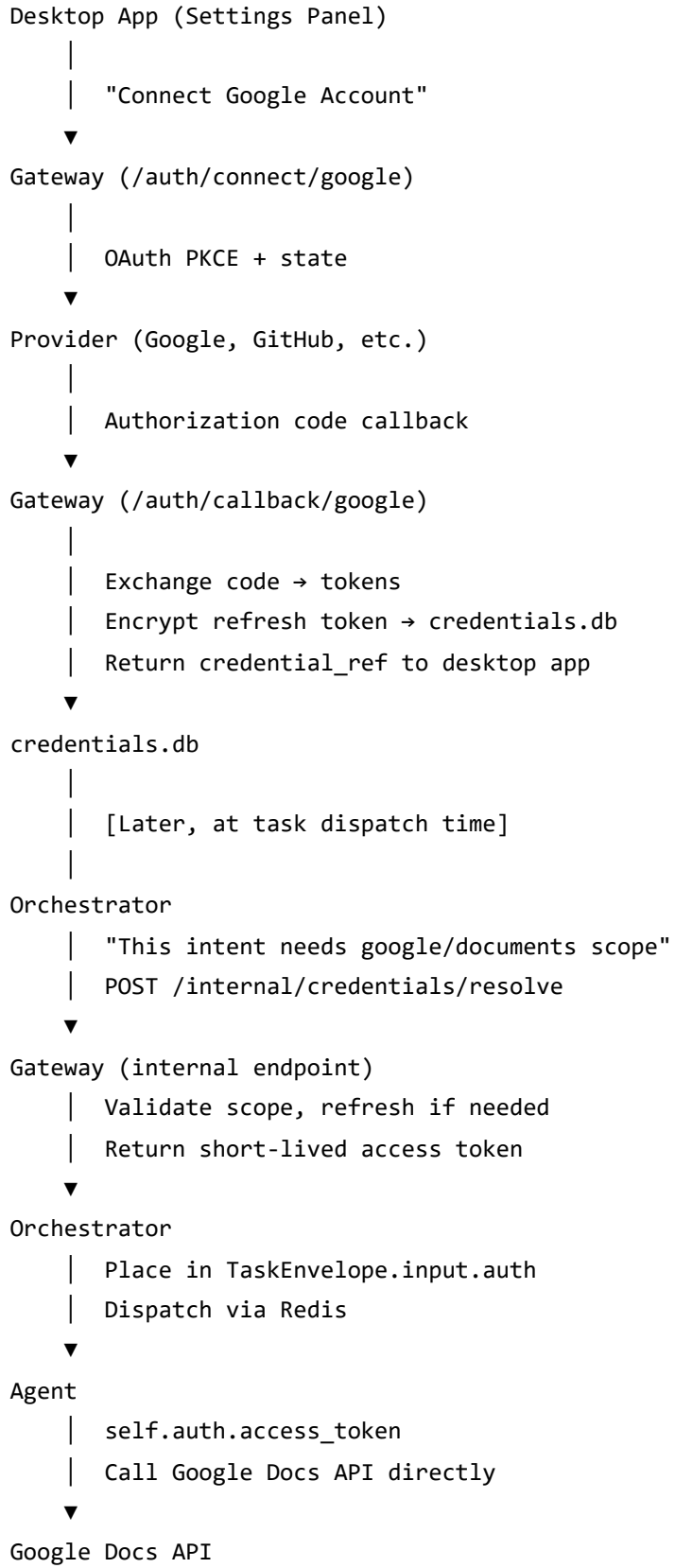
1. Merge Phase 1 config. Deploy to all nodes. Verify no parse errors in logs.
2. Merge Phase 2 config. Deploy to all nodes. Verify warnings (not errors) in logs.
3. Once warning volume drops to zero: merge Phase 3 config. Deploy.
4. Done — cluster is fully on new version.

22. Credential Management

The Credential Manager is a module inside the Gateway that handles user-connected provider accounts (Google, GitHub, etc.), OAuth token lifecycle, and secure credential resolution for agents. It is the single owner of all user OAuth credentials in the system.

Design principle: Agents call external provider APIs directly using short-lived access tokens passed in `TaskEnvelope.input.auth`. Refresh tokens never leave the Gateway. The orchestrator resolves credentials at dispatch time via an internal Gateway endpoint. No proxy layer — agents are first-party code on the same VM, and the trust boundary is the VM itself.

22.1 Architecture Overview



22.2 Data Model

All credential data lives in `gateway/credentials.db` — separate from `sessions.db` for security isolation. This database **must** be on a persistent volume in containerized deployments.


```

-- gateway/credentials.db

-- Connected provider accounts
CREATE TABLE accounts (
    account_id TEXT PRIMARY KEY,          -- 'acc_a1b2c3'
    user_id TEXT NOT NULL,                -- links to user (single-user for now)
    provider TEXT NOT NULL,               -- 'google', 'github', 'microsoft'
    provider_account_id TEXT NOT NULL,    -- provider's own account ID
    email TEXT,                           -- display email for account selection UI
    display_name TEXT,                    -- 'Work Gmail', 'Personal Google'
    is_primary BOOLEAN DEFAULT FALSE,     -- default account for this provider
    status TEXT DEFAULT 'active',         -- 'active', 'revoked', 'expired'
    connected_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    UNIQUE (user_id, provider, provider_account_id)
);

-- Encrypted credentials per account
CREATE TABLE credentials (
    credential_ref TEXT PRIMARY KEY,      -- 'cred_9f3a...' - opaque reference
    account_id TEXT NOT NULL,
    granted_scopes TEXT NOT NULL,         -- JSON array of granted OAuth scopes
    encrypted_refresh_token BLOB NOT NULL, -- Fernet-encrypted refresh token
    access_token_cache TEXT,              -- cached access token (encrypted)
    access_token_expires_at TIMESTAMP,    -- when cached access token expires
    revoked_at TIMESTAMP,                 -- set on disconnect/revocation
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    FOREIGN KEY (account_id) REFERENCES accounts(account_id)
);

-- Resource-to-account bindings (learned over time)
CREATE TABLE resource_bindings (
    binding_id INTEGER PRIMARY KEY AUTOINCREMENT,
    resource_type TEXT NOT NULL,          -- 'google_doc', 'github_repo', etc.
    external_id TEXT NOT NULL,            -- provider's resource ID
    display_name TEXT,                    -- 'Project Proposal', 'cosmic-os'
    account_id TEXT NOT NULL,
    last_accessed_at TIMESTAMP,
    FOREIGN KEY (account_id) REFERENCES accounts(account_id),
    UNIQUE (resource_type, external_id, account_id)
);

CREATE INDEX idx_bindings_name ON resource_bindings(resource_type, display_name);

```

```
-- Credential usage audit log
CREATE TABLE credential_audit (
    audit_id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TIMESTAMP NOT NULL,
    task_id TEXT NOT NULL,
    agent_id TEXT NOT NULL,
    credential_ref TEXT NOT NULL,
    provider TEXT NOT NULL,
    scopes_used TEXT NOT NULL,          -- JSON array of scopes used
    action TEXT NOT NULL,               -- 'resolve', 'refresh', 'revoke'
    result TEXT NOT NULL,               -- 'success', 'failed', 'expired'
    FOREIGN KEY (credential_ref) REFERENCES credentials(credential_ref)
);

CREATE INDEX idx_audit_task ON credential_audit(task_id);
CREATE INDEX idx_audit_credential ON credential_audit(credential_ref);
```

22.3 Orchestrator Credential Resolution (Dispatch-Time)

When the orchestrator decomposes a user request into sub-tasks, it checks each target agent's `agent_card.yaml` for `auth_requirements` (see §6.2). If the intent requires provider credentials, the orchestrator resolves them before dispatch.

Orchestrator dispatch flow (updated from §10):

```

async def dispatch_to_agent(self, intent: str, input_data: dict,
                           session_context: dict, redis):
    # 1. Look up routing.yaml → target agent_id
    routing = self.routing_config[intent]
    agent_id = routing['agent']

    # 2. Check agent_card for auth_requirements
    card = self.registry.get_card(agent_id)
    auth_req = card.get('auth_requirements', {}).get(intent)

    if auth_req:
        # 3. Resolve account + credential
        credential = await self._resolve_credential(
            auth_req, session_context, input_data
        )
        if credential is None:
            # No account connected or ambiguous – escalate to user
            return await self._escalate_account_connection(
                auth_req, session_context
            )
        input_data['auth'] = credential

    # 4. Build and dispatch TaskEnvelope as normal
    task = TaskEnvelope(
        intent=intent,
        input=input_data,
        recipient=agent_id,
        ...
    )
    await dispatch(task, redis)

async def _resolve_credential(self, auth_req: dict,
                             session_context: dict,
                             input_data: dict) -> dict | None:
    """Call Gateway internal API to resolve credential."""
    resp = await self.gateway_client.post(
        f'{GATEWAY_INTERNAL_URL}/internal/credentials/resolve',
        json={
            'provider': auth_req['provider'],
            'required_scopes': auth_req['scopes'],
            'user_id': session_context['user_id'],
            'account_id': session_context.get('account_id'), # if user specified
            'resource_hint': input_data.get('doc_id') or input_data.get('repo'),
        },
        headers={'X-Internal-Token': GATEWAY_INTERNAL_TOKEN},
    )

```

```

)
if resp.status_code == 200:
    return resp.json()
    # Returns: { credential_ref, access_token, provider, scopes, expires_at }
elif resp.status_code == 404:
    return None # no matching account/credential
elif resp.status_code == 403:
    return None # scopes not granted – needs re-consent
else:
    raise CredentialResolveError(resp.text)

```

22.4 Account Resolution Policy

When the orchestrator needs to determine which provider account to use, it follows a deterministic policy. The orchestrator owns this decision — agents never guess accounts.

Resolution order (first match wins):

Priority	Condition	Action
1	User explicitly names account ("use my work Gmail")	Orchestrator extracts account hint from intent parsing, passes <code>account_id</code> to resolve
2	User names a resource ("update Project Proposal doc")	Orchestrator dispatches <code>{agent}.resolve_resource</code> intent to the agent (e.g., <code>docs.resolve_resource</code>). Agent searches across connected accounts, returns match + account binding.
3	Session context has an active account for this provider	Use the account from the current session's last successful provider interaction
4	Exactly one connected account for the provider	Use it — no ambiguity
5	Multiple connected accounts, ambiguous	Orchestrator escalates to user via <code>user.input_required</code> with the list of connected accounts
6	No account connected for the provider	Orchestrator escalates to user via <code>user.input_required</code> asking to connect an account

Resource resolution example (Priority 2):

```
# User says: "Update the Project Proposal doc"
# Orchestrator doesn't know which account or doc_id

# Step 1: Dispatch resolve_resource to docs_agent
resolve_task = TaskEnvelope(
    intent='docs.resolve_resource',
    input={
        'query': 'Project Proposal',
        'resource_type': 'google_doc',
        'auth': all_google_credentials, # tokens for ALL connected accounts
    },
    recipient='cosmic/docs-agent:2.1.0',
    ...
)

# Step 2: docs_agent searches Google Drive across all accounts
# Returns: { matches: [{ doc_id, title, account_id, modified_at }] }

# Step 3: If exactly 1 match → proceed with that account + doc_id
#         If multiple → orchestrator asks user (shows title + account + date)
#         If none → orchestrator tells user "document not found"

# Step 4: Gateway stores the binding in resource_bindings table
#         Next time, Priority 2 resolves instantly via lookup
```

22.5 Token Refresh Mid-Task

Access tokens are short-lived (5-15 minutes). Most tasks complete within this window. If a task runs longer and the token expires, the agent uses the existing suspension mechanism (§13.1):

```

# Inside agent's execute() method – provider API call fails with 401
async def execute(self, task: TaskEnvelope):
    try:
        result = await self.call_provider_api(...)
    except ProviderAuthError:
        # Token expired mid-task – request refresh via orchestrator
        await self._send_reverse_task(
            intent='orchestrator.refresh_credential',
            input={
                'credential_ref': self.auth['credential_ref'],
                'provider': self.auth['provider'],
                'parent_task_id': task.task_id,
            },
        )
        # Agent suspends (task.suspended event emitted)
        # State serialized to runtime/state.db
        # Orchestrator calls Gateway to refresh, gets new token
        # Orchestrator resumes agent with:
        #   intent: 'agent.resume'
        #   input: { task_id, auth: { new access_token, ... } }
        return # will be resumed

```

Orchestrator-side refresh handling:

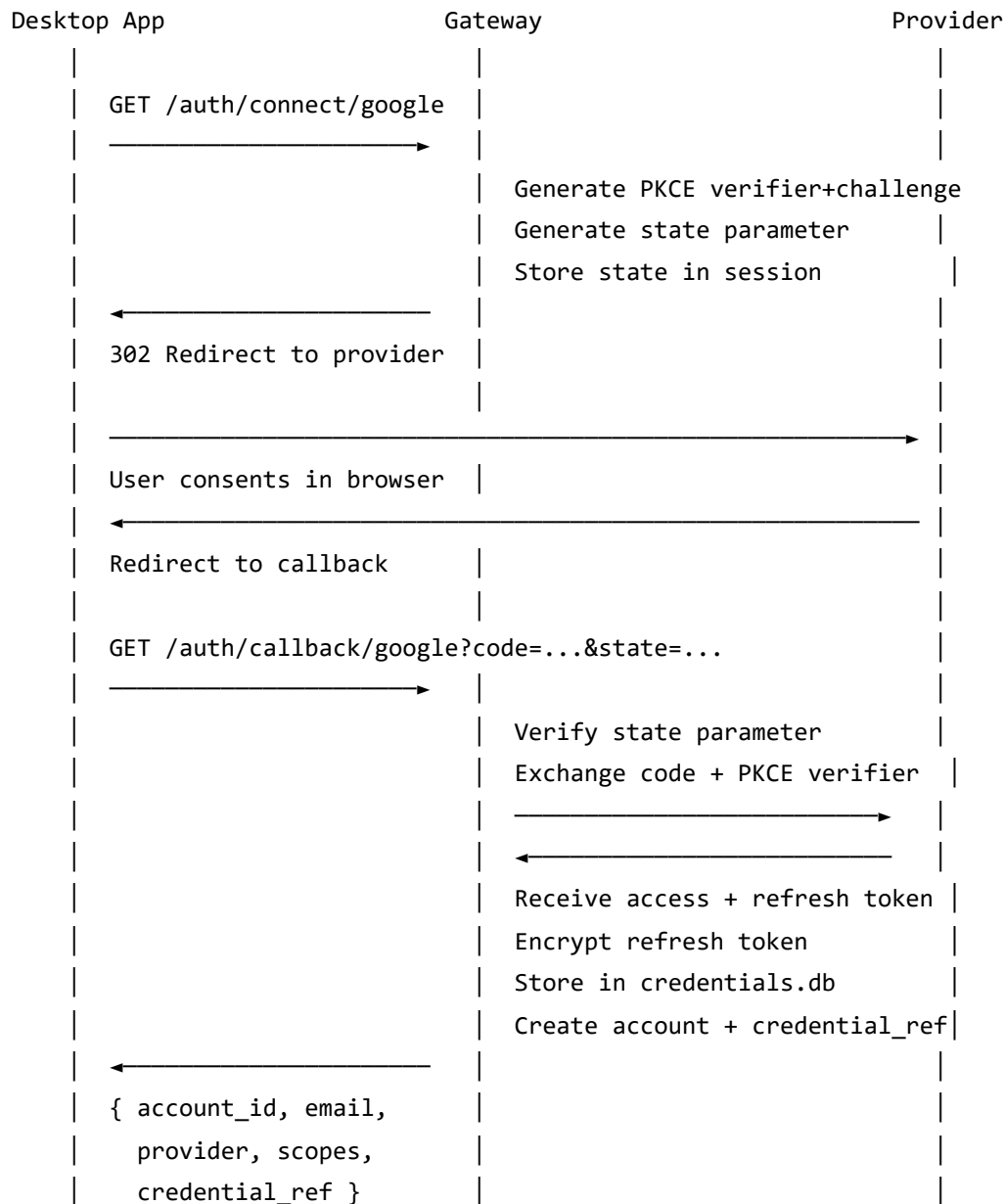
```

async def on_refresh_credential(self, event):
    credential_ref = event.input['credential_ref']
    # Call Gateway internal endpoint – it handles the refresh
    new_auth = await self.gateway_client.post(
        f'{GATEWAY_INTERNAL_URL}/internal/credentials/resolve',
        json={
            'credential_ref': credential_ref,
            'force_refresh': True,
        },
        headers={'X-Internal-Token': GATEWAY_INTERNAL_TOKEN},
    )
    # Resume agent with fresh token
    await self._resume_agent(
        task_id=event.input['parent_task_id'],
        input={'auth': new_auth.json()},
    )

```

22.6 OAuth Flow

OAuth flows are initiated from the desktop app's settings panel and handled entirely by the Gateway.



Provider adapter pattern: Each provider (Google, GitHub, Microsoft) has a small adapter that knows the provider's OAuth endpoints, scopes format, and token refresh mechanism. New providers are added by implementing the adapter interface.

```
# gateway/credentials/providers.py
class ProviderAdapter:
    """Base class for OAuth provider adapters."""
    provider: str
    authorize_url: str
    token_url: str
    revoke_url: str | None

    def get_authorize_params(self, scopes: list[str], state: str,
                           code_challenge: str) -> dict: ...
    def exchange_code(self, code: str, code_verifier: str) -> TokenResponse: ...
    def refresh_token(self, refresh_token: str) -> TokenResponse: ...
    def revoke_token(self, token: str) -> bool: ...

class GoogleAdapter(ProviderAdapter):
    provider = 'google'
    authorize_url = 'https://accounts.google.com/o/oauth2/v2/auth'
    token_url = 'https://oauth2.googleapis.com/token'
    revoke_url = 'https://oauth2.googleapis.com/revoke'
    ...

class GitHubAdapter(ProviderAdapter):
    provider = 'github'
    authorize_url = 'https://github.com/login/oauth/authorize'
    token_url = 'https://github.com/login/oauth/access_token'
    revoke_url = None # GitHub doesn't support programmatic revocation
    ...
```

22.7 Token Encryption

Refresh tokens are encrypted at rest using Fernet symmetric encryption. The encryption key is loaded from the `CREDENTIAL_ENCRYPTION_KEY` environment variable.


```
# gateway/credentials/encryption.py
from cryptography.fernet import Fernet

FERNET_KEY = os.environ['CREDENTIAL_ENCRYPTION_KEY']
cipher = Fernet(FERNET_KEY)

def encrypt_token(plaintext: str) -> bytes:
    return cipher.encrypt(plaintext.encode('utf-8'))

def decrypt_token(ciphertext: bytes) -> str:
    return cipher.decrypt(ciphertext).decode('utf-8')
```

Why Fernet: Simple, authenticated encryption (AES-128-CBC + HMAC-SHA256). No IV/nonce management. Built into the `cryptography` library. Sufficient for a single-VM deployment. If you move to multi-node, swap to a KMS envelope encryption pattern.

22.8 Revocation & Disconnect

When a user disconnects a provider account from the settings panel:

1. Gateway calls the provider's revoke endpoint (if supported).
2. Gateway marks the credential as `revoked` in `credentials.db` (`revoked_at = now()`).
3. Gateway marks the account as `revoked` in `accounts` table.
4. Any future `resolve` calls for this `credential_ref` return `403 REVOKED` .
5. In-flight tasks using this credential will fail with `AUTH_ERROR` (non-retryable per §19.1) on their next provider API call.

```

# gateway/credentials/manager.py
async def disconnect_account(account_id: str):
    account = db.execute(
        'SELECT * FROM accounts WHERE account_id = ?', [account_id]
    ).fetchone()

    creds = db.execute(
        'SELECT * FROM credentials WHERE account_id = ? AND revoked_at IS NULL',
        [account_id]
    ).fetchall()

    for cred in creds:
        # Attempt provider-side revocation (best-effort)
        adapter = get_provider_adapter(account['provider'])
        if adapter.revoke_url:
            refresh_token = decrypt_token(cred['encrypted_refresh_token'])
            adapter.revoke_token(refresh_token)

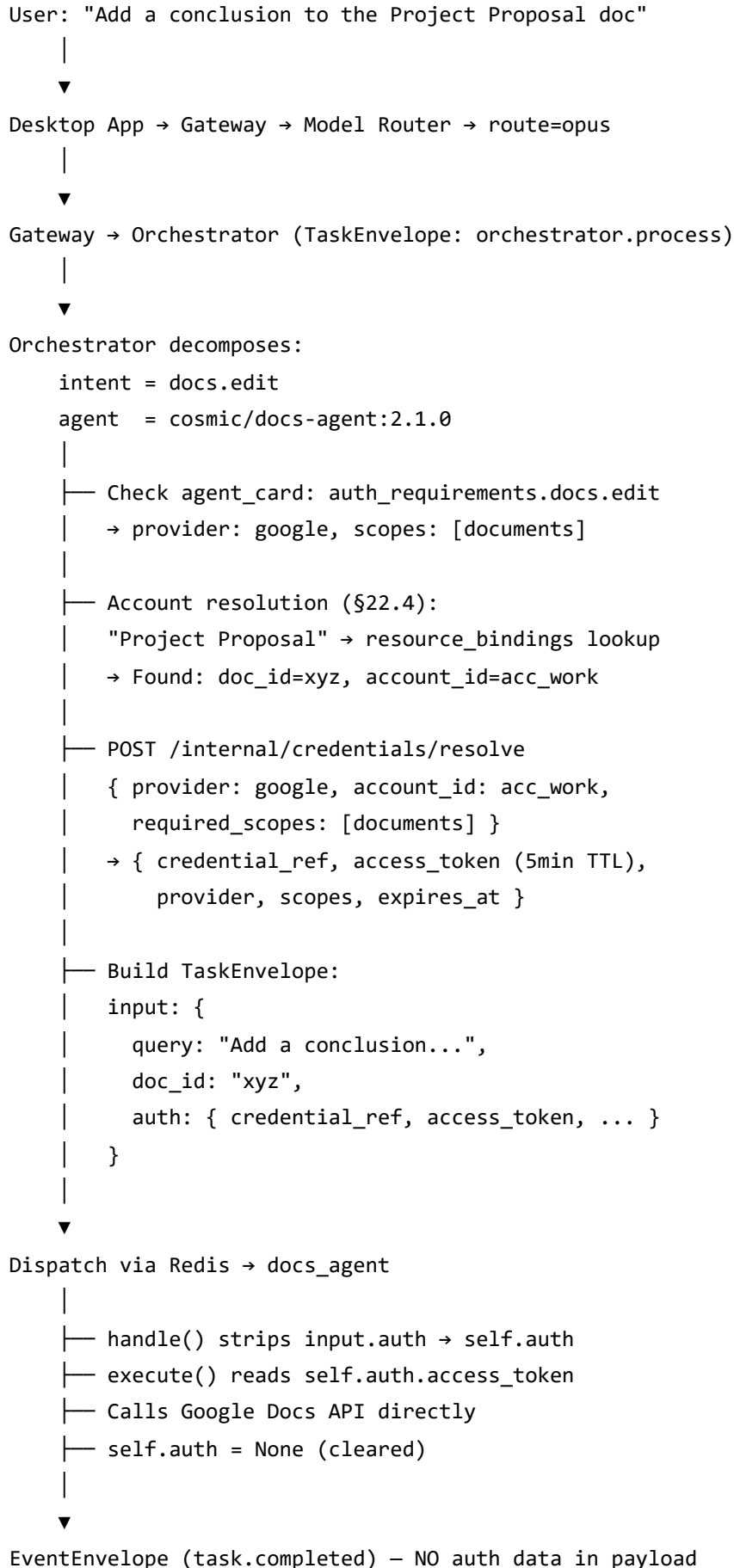
        # Mark revoked in DB
        db.execute('''
            UPDATE credentials SET revoked_at = ?, updated_at = ?
            WHERE credential_ref = ?
            ''', [utcnow(), utcnow(), cred['credential_ref']])

    db.execute('''
        UPDATE accounts SET status = 'revoked', updated_at = ?
        WHERE account_id = ?
        ''', [utcnow(), account_id])

    # Audit
    db.execute('''
        INSERT INTO credential_audit
        (timestamp, task_id, agent_id, credential_ref, provider, scopes_used, action, result)
        VALUES (?, 'system', 'system', ?, ?, '[]', 'revoke', 'success')
        ''', [utcnow(), creds[0]['credential_ref'] if creds else 'none', account['provider']])

```

22.9 Credential Flow Diagram (Full Runtime)





Orchestrator → Gateway → Desktop App

"Done – conclusion added to Project Proposal"

22.10 Hard Rules

These rules are non-negotiable. Violating any of them is a security incident.

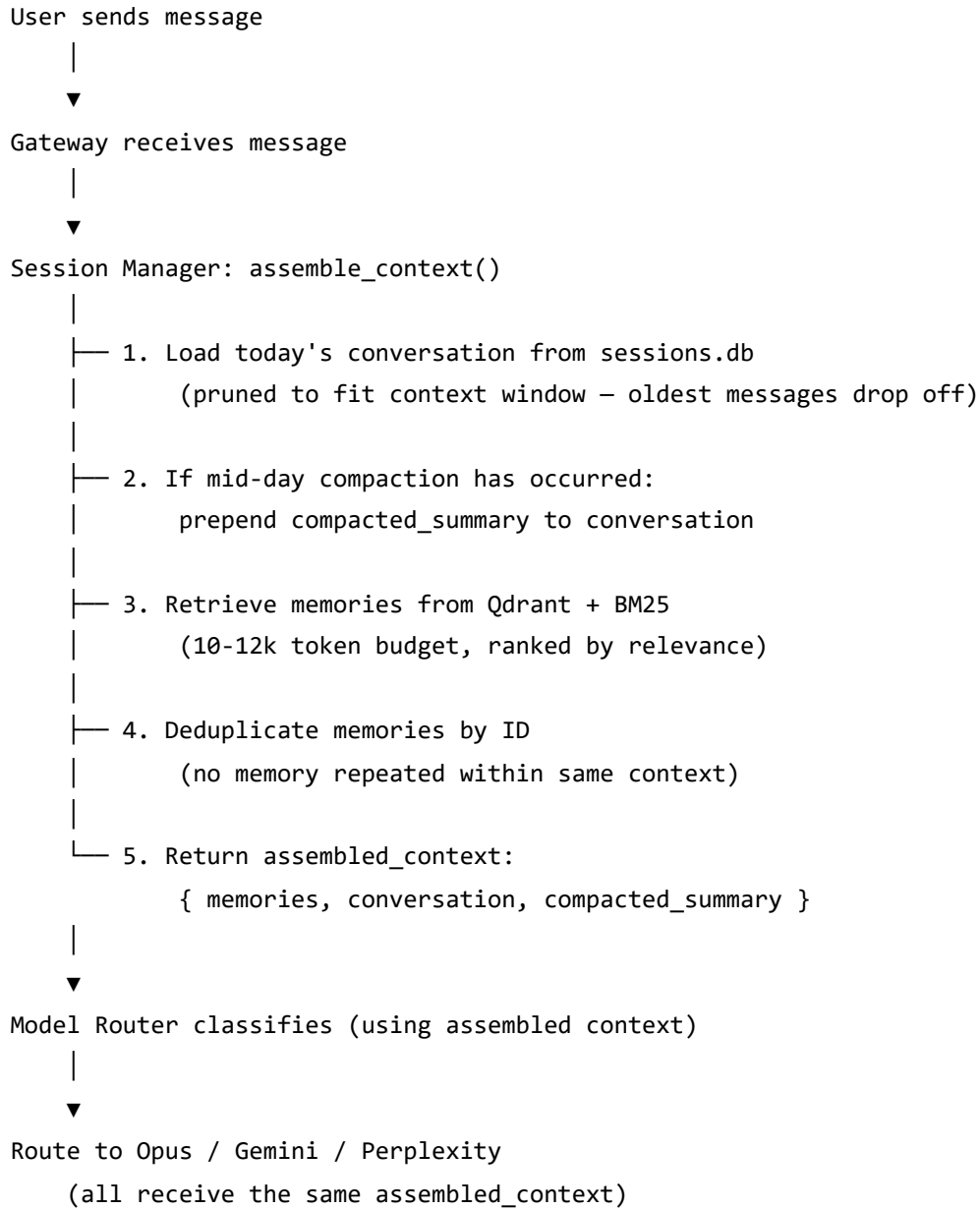
1. **Refresh tokens never leave `credentials.db`**. Only the Gateway Credential Manager reads them. Not in Redis, not in TaskEnvelopes, not in events, not in logs.
2. **Access tokens only appear in `TaskEnvelope.input.auth`**. The agent base class strips this field before any serialization (see §12.6). Never in EventEnvelopes, artifacts, `store/`, `runtime/`, or `learnings.md`.
3. **`credential_audit` logs `credential_ref`, never token values**. Audit by reference, not by secret.
4. **Agents never guess accounts**. Account resolution is orchestrator-owned (§22.4). If ambiguous, the orchestrator asks the user.
5. **Revocation is immediate**. When a user disconnects an account, the credential is marked revoked in the DB. Future resolve calls fail. In-flight tasks fail on next provider API call.
6. **OAuth client secrets are env vars**. Never in source code, never in `credentials.db`, never in agent folders.
7. **One `credential_ref` per (account, agent, scope-set)**. No shared "global token" across agents. If two agents need the same provider, each gets its own `credential_ref` with its own scope validation.

23. Session & Memory Management

The Session Manager is a module inside the Gateway that creates a perpetual conversational experience. It assembles context for every query by combining today's conversation with retrieved long-term memories, manages daily session lifecycles, and handles context window pruning and compaction.

Design principle: Three separate memory layers with different lifecycles — today's conversation (short-term, compactable), retrieved memories (long-term, never compacted, retrieved fresh each turn), and task execution (isolated, retrievable on demand). All LLM backends (Opus, Gemini, Perplexity) receive the same assembled context — the user gets a consistent assistant regardless of which model answers.

23.1 Architecture Overview



23.2 Session Lifecycle

Each day is a session. The user sees one perpetual conversation — session boundaries are transparent.

	4:00 AM	4:00 AM
Session A	Session B (today)	Session C
At boundary:		
1. Force compact		
Session A		
2. Write summary		
as memory		
→ memory/		
sessions/		
→ Qdrant		
3. Start fresh		
Session B		

Session lifecycle:

Event	What Happens
App startup	Gateway loads or creates today's session. Desktop app receives the tail of today's conversation history for display.
Each message	Session Manager assembles context (conversation + memories). Message and response stored in <code>sessions.db</code> .
Context at 70%	Mid-day compaction triggers: older messages summarized, summary replaces them in context. Conversation continues with <code>[compacted summary] + [recent messages]</code> .
4:00 AM	Forced compaction of remaining session. Compacted summary written as a memory to <code>memory/sessions/</code> + Qdrant. New session ID created.

23.3 Context Window Management

Two mechanisms keep the context window under control:

Pruning (continuous): As new messages arrive, the oldest messages in the context window are pruned — they stop being sent to the LLM. Pruned messages remain in `sessions.db` for full history browsing in the desktop app UI. Pruning is lightweight and always running.

Compaction (triggered): When context usage hits 70% of the target model's context window, compaction runs:

1. Extract memories from the conversation before summarizing (important facts, decisions, outcomes are written to the memory store at full fidelity)

2. Send the older conversation messages to a fast/cheap model (Gemini Flash or Sonnet — not Opus) with a summarization prompt
3. The summary replaces the older messages in the session context
4. Conversation continues with: [compacted summary] + [recent messages since compaction]

gateway/session/compaction.py

```

async def check_and_compact(session_id: str, context_tokens: int,
                           model_context_window: int):
    threshold = model_context_window * 0.70
    if context_tokens < threshold:
        return # no compaction needed

    # Step 1: Extract memories before compacting
    messages = await get_compactable_messages(session_id)
    await extract_and_store_memories(messages)

    # Step 2: Summarize via cheap/fast model (NOT Opus)
    summary = await compaction_llm.summarize(
        messages=messages,
        instruction='Summarize this conversation preserving key decisions, '
                    'facts, user preferences, and action items.',
        max_output_tokens=4000,
    )

    # Step 3: Replace messages with summary in session
    await store_compacted_summary(session_id, summary)
    db.execute('''
        UPDATE sessions SET compaction_count = compaction_count + 1,
        compacted_summary = ?, updated_at = ?
        WHERE session_id = ?
    ''', [summary, utcnow(), session_id])

```

Token budget allocation per LLM call:

Total context window (model-specific)

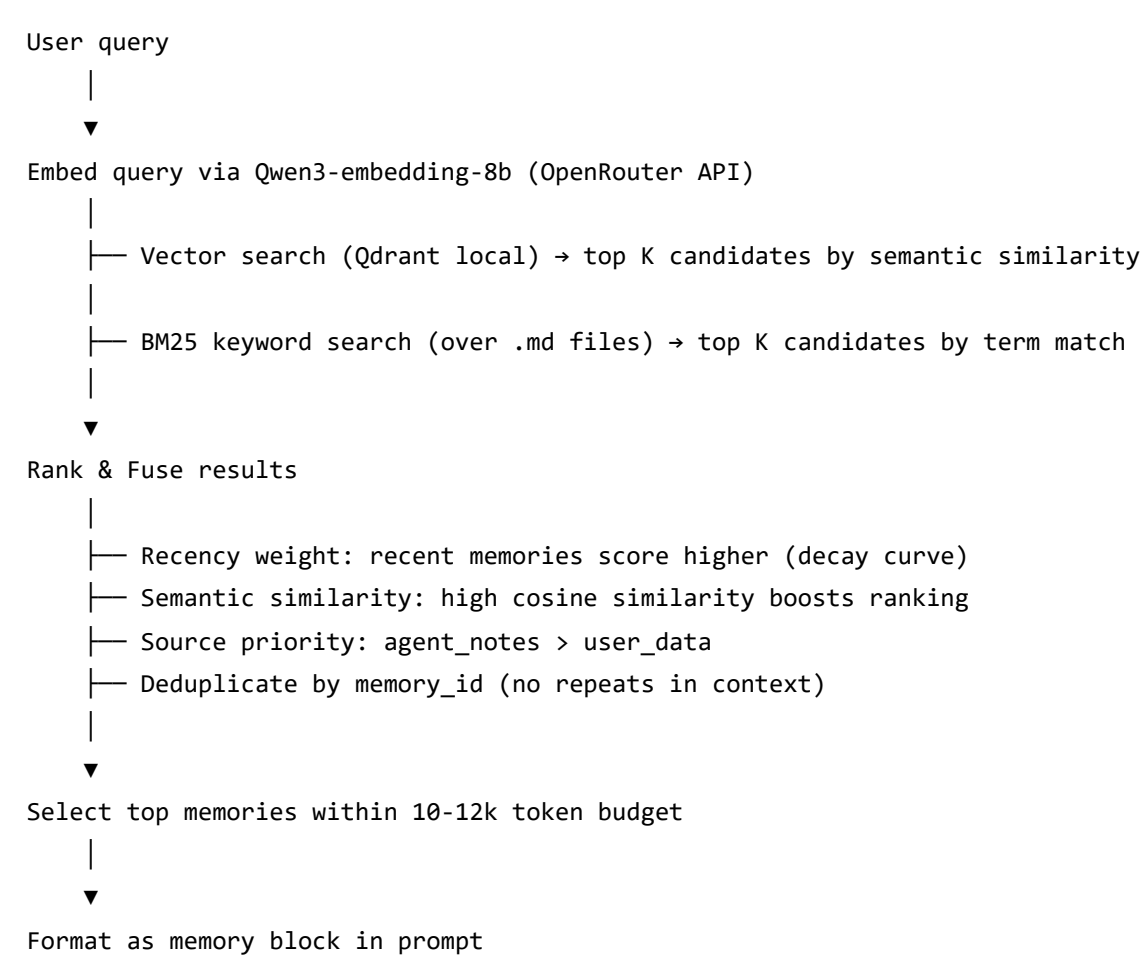
- |
- ├─ System prompt: ~1-2k tokens
- ├─ Retrieved memories: 10-12k tokens (fixed budget)
- ├─ Compaction reserve: ~4-6k tokens (space for compaction output)
- ├─ Output reserve: ~4k tokens (for model's response)
- └─ Today's conversation: fills remaining space
 - └─ Compaction triggers at 70% of THIS allocation

Why a compaction reserve? The compaction process itself produces a summary (2-4k tokens) that replaces the older messages. The reserve ensures there's space for this summary plus headroom for incoming messages while compaction runs. This matches the pattern used by Claude Code's auto-compression.

23.4 Memory Retrieval

On every turn, the Session Manager retrieves relevant memories from the long-term store using hybrid search. Memories are assembled fresh each turn — they are NOT part of the conversation that gets compacted.

Retrieval pipeline:



Memory types and their priority:

Type	Source	Priority	Description
Compacted session summaries	memory/sessions/*.md	Normal	Past day summaries — broad conversational context

Type	Source	Priority	Description
Agent notes	memory/agent_notes/*/learnings.md	High	Agent-curated facts: user preferences, task-specific notes, learned patterns
User data	memory/user_data/	Normal	Indexed emails, files, documents — provides grounding
Task summaries	memory/tasks/*.md	Normal	Past task results — retrievable context about completed work

Agent notes are prioritized over user data. Agent notes are curated, high-signal information that the system has already processed and found important. Raw user data is bulk context that needs relevance ranking to surface the right pieces.

Critical rule: memories are excluded from compaction input. When compaction runs, it only sees the raw conversation messages. The retrieved memories block is stripped before the compaction LLM call. This prevents a feedback loop: memory retrieved → included in context → compacted into summary → that summary becomes a memory → retrieved again → compacted again. Memories persist at source fidelity and are re-retrieved each turn based on relevance.

23.5 Memory Storage: .md Files as Source of Truth

All memories are stored as `.md` files in the `memory/` directory tree. Qdrant is the **index**, not the source of truth. If Qdrant data is lost, it is rebuilt by re-embedding the `.md` files.

File format:

```
<!-- memory/sessions/2025-01-15.md -->
---
memory_id: mem_sess_20250115
type: session_summary
date: 2025-01-15
token_count: 1850
tags: [architecture, oauth, credential-management]
---
```

Session Summary – January 15, 2025

User worked on the COSMIC multi-agent architecture. Key decisions:

- Credential Manager lives inside the Gateway as a module
- Agents receive short-lived access tokens in TaskEnvelope.input.auth
- Refresh tokens never leave the Gateway
- Account resolution is orchestrator-owned

User preferences noted:

- Prefers simple solutions over enterprise patterns
- Wants implementation-ready specs, not abstract descriptions

```
<!-- memory/agent_notes/docs_agent/learnings.md -->
---
memory_id: mem_agent_docs_001
type: agent_note
agent_id: cosmic/docs-agent:2.1.0
updated_at: 2025-01-15
tags: [user-preference, formatting]
---
```

Docs Agent – Learnings

User Preferences

- User prefers APA citation format
- User wants section headings in title case
- Default Google account for docs: work@company.com

```
<!-- memory/tasks/tsk_abc123.md -->
---
memory_id: mem_task_tsk_abc123
type: task_summary
task_id: tsk_abc123
agent_ids: [cosmic/research-agent:1.0.0, cosmic/docs-agent:2.1.0]
completed_at: 2025-01-15T14:30:00Z
tags: [research, quantum-computing]
---
```

Task: Research quantum computing advances

Result

Found 5 papers on recent quantum error correction breakthroughs.
Summary delivered to user. Key sources: arxiv:2401.xxxxx, arxiv:2401.yyyyy.

Artifacts

- runs/artifacts/tsk_abc123/summary.md
- runs/artifacts/tsk_abc123/citations.json

Memory write pipeline:

```
# gateway/session/memory_writer.py

async def write_memory(memory_id: str, memory_type: str, content: str,
                      metadata: dict):
    # 1. Write .md file (source of truth)
    path = get_memory_path(memory_type, metadata)
    write_md_file(path, memory_id, memory_type, metadata, content)

    # 2. Embed and index in Qdrant
    embedding = await embed_text(content) # Qwen3-embedding-8b via OpenRouter
    qdrant.upsert(
        collection_name='memories',
        points=[{
            'id': memory_id,
            'vector': embedding,
            'payload': {
                'type': memory_type,
                'date': metadata.get('date'),
                'tags': metadata.get('tags', []),
                'path': str(path),
                'token_count': count_tokens(content),
            },
        }],
    )
```

23.6 Task Memory Isolation

Task execution happens in independent sessions. The full back-and-forth of task execution (agent progress, intermediate results, retries, internal clarifications) does NOT enter the main conversation session. Only the final task result is inserted as a message in the main session.

Why isolation? A task like "research quantum computing" might involve the research agent making 10 web searches, evaluating sources, reading papers, and synthesizing findings. All of that internal chatter would overwhelm the main session. The user sees: "Here's what I found about quantum computing: [summary]" — clean and focused.

Where task memories live:

Data	Location	Accessible By
Task execution events	streams:events (Redis)	Orchestrator, Gateway (real-time forwarding to UI)

Data	Location	Accessible By
Agent session data	agents/*/store/data/ (per-agent SQLite)	The agent that owns it, via recall intents
Agent learnings	agents/*/store/learnings.md → synced to memory/agent_notes/	Session Manager (retrieved as high-priority memories)
Task result summary	memory/tasks/<task_id>.md	Session Manager (retrieved when relevant)
Task artifacts	runs/artifacts/<task_id>/	Any agent via ArtifactManifest

How the orchestrator retrieves task memories:

The orchestrator has two paths for accessing past task context:

Path 1: Event stream replay. The orchestrator's consumer group on `streams:events` has the full event history for all tasks. It can reconstruct the event timeline for any `task_id` by querying the stream. This gives the orchestrator the sequence of events (accepted, progress, completed/failed) and their payloads.

```
# Orchestrator retrieves task event history
async def get_task_events(task_id: str, redis) -> list[EventEnvelope]:
    """Scan streams:events for all events matching this task_id."""
    events = []
    results = await redis.xrange('streams:events', '-', '+')
    for msg_id, data in results:
        event = EventEnvelope.model_validate_json(data['event'])
        if event.task_id == task_id:
            events.append(event)
    return sorted(events, key=lambda e: e.seq)
```

Path 2: Recall intents to agents. For detailed agent-specific history (what sources were evaluated, what edits were made, what decisions were taken), the orchestrator dispatches recall intents to the relevant agents. Each agent queries its own `store/data/` and returns structured results. This is already designed in §12.4.

```

# Orchestrator asks docs_agent about a past task
recall_task = TaskEnvelope(
    intent='docs.recall_session',
    input={
        'task_id': 'tsk_abc123',
        'query': 'what edits were made and which account was used',
    },
    recipient='cosmic/docs-agent:2.1.0',
    ...
)
# docs_agent queries its own store/data/sessions.db
# Returns: structured edit history with doc_id, account, changes

```

How task memories enter the retrieval store:

After task completion, the orchestrator writes a task summary to `memory/tasks/<task_id>.md`. This summary includes: what was requested, which agents were involved, the final result, and any artifacts produced. This file is embedded and indexed in Qdrant. When the user later asks "what did we do about quantum computing?", the memory retriever finds this task summary via hybrid search and includes it in the assembled context.

```

# Orchestrator writes task summary after completion
async def on_task_completed(task_id: str, result: AgentResult,
                           original_request: str, agents_used: list[str]):
    summary = await compaction_llm.summarize(
        f'Task: {original_request}\n'
        f'Agents: {"", ".join(agents_used)}\n'
        f'Result: {json.dumps(result.output)}\n'
        f'Artifacts: {[a.path for a in result.artifacts]}\n',
        instruction='Write a brief summary of this completed task.',
        max_output_tokens=500,
    )
    await memory_writer.write_memory(
        memory_id=f'mem_task_{task_id}',
        memory_type='task_summary',
        content=summary,
        metadata={
            'task_id': task_id,
            'agent_ids': agents_used,
            'completed_at': utcnow().isoformat(),
        },
    )

```

23.7 Agent Notes Sync

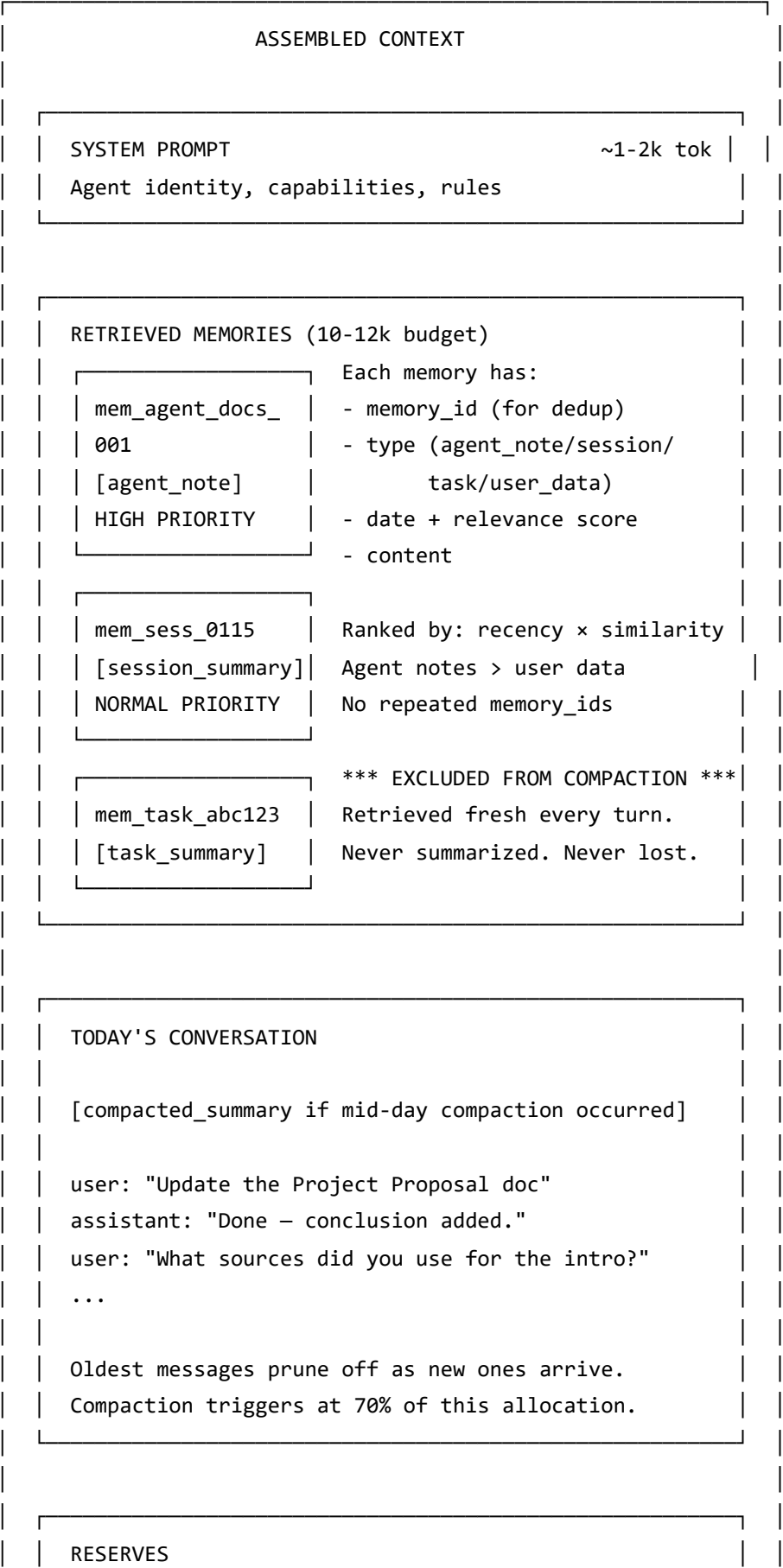
Agent learnings (`agents/*/store/learnings.md`) are the agents' own persistent memories (see §12.1). The Session Manager syncs these into `memory/agent_notes/` and indexes them in Qdrant so they're retrievable as high-priority memories.

Sync mechanism: After each task completion, the Session Manager checks if the agent updated its `learnings.md` (by comparing file hash). If changed, it re-indexes the file in Qdrant and copies it to `memory/agent_notes/`. Agents continue to own their learnings files — the sync is read-only from the Session Manager's perspective.

23.8 Configuration

```
# Session Manager configuration (gateway environment)
SESSION_RESET_HOUR=4                # daily reset at 4:00 AM local time
COMPACTION_THRESHOLD=0.70           # compact at 70% context usage
COMPACTION_MODEL=gemini-flash        # cheap/fast model for summarization
MEMORY_TOKEN_BUDGET=12000           # max tokens for retrieved memories
MEMORY_MAX_RESULTS=20               # max memories per retrieval
EMBEDDING_MODEL=qwen3-embedding-8b  # via OpenRouter
OPENROUTER_API_KEY=<secret>          # for embedding API calls
QDRANT_PATH=./qdrant_data            # local Qdrant storage path
MEMORY_STORE_PATH=./memory          # .md file tree root
```

23.9 Context Assembly Diagram (Per Turn)



Compaction reserve: ~4-6k	Output reserve: ~4k
---------------------------	---------------------

23.10 Hard Rules

- 1. **All LLM backends receive the same assembled context.** Whether the query routes to Opus, Gemini, or Perplexity, the Session Manager provides identical context. The user gets a consistent assistant.
- 2. **Memories are never compacted.** The memory block is stripped before compaction input. Memories persist at source fidelity and are re-retrieved each turn.
- 3. **Each memory has a unique `memory_id`.** No memory is repeated within the same context window. The Session Manager tracks which IDs are in context.
- 4. **Task execution is isolated from the main session.** Only final task results enter the main session as messages. Full task details are retrievable via recall intents and the memory store.
- 5. **Agent notes have priority over user data** in memory ranking. Curated, task-proven facts outrank bulk external data.
- 6. **.md files are the source of truth.** Qdrant is the index. If Qdrant is lost, rebuild from `memory/`.
- 7. **Compaction uses a cheap model.** Never use Opus for summarization. Gemini Flash or Sonnet — fast, cheap, good enough.
- 8. **Daily reset is transparent to the user.** Session boundaries at 4AM are invisible. The compacted summary becomes a retrievable memory. The user sees one perpetual conversation.

Appendix A: Quick Reference — All Redis Keys

Key Pattern	Type	Scope	TTL
<code>streams:{agent_id}:{priority}</code>	Stream	Per-agent	Persistent (trimmed by MAXLEN)
<code>streams:events</code>	Stream	Global	Persistent
<code>streams:dlq</code>	Stream	Global	Persistent
<code>streams:capability.updates</code>	Stream	Global	Persistent
<code>streams:broadcast</code>	Stream	Global	Persistent
<code>registry:{agent_id}:{instance_id}</code>	Hash	Per-instance	<code>heartbeat_ttl_sec + 5</code> (auto-expired)

Key Pattern	Type	Scope	TTL
intent:{intent_name}	Set	Per-intent	Registration-driven
idempotency:{idempotency_key}	String	Per-task	2 × deadline or 2 × max_task_duration_sec (floor 60s)
idempotency:result:{idempotency_key}	String	Per-task	RESULT_TTL_SEC (7 days)
event_seq:{task_id}	Counter	Per-task	RESULT_TTL_SEC after terminal
orchestrator:leader	String	Global	LEADER_TTL (15s)
orchestrator:epoch	Counter	Global	Persistent

Consumer groups on streams:events :

Group	Consumer	Purpose
orchestrator	Orchestrator instances	Routing, retry, DLQ, deferred recovery
gateway	Gateway instances	Forward events to desktop app via WebSocket

Appendix B: Quick Reference — All Pydantic Models

Model	Defined In	Direction
TaskEnvelope	shared/contracts.py	Orchestrator → Agent
EventEnvelope	shared/contracts.py	Agent → Orchestrator
AgentResult	shared/contracts.py	Agent → Orchestrator (terminal)
AgentError	shared/contracts.py	Inside AgentResult
TaskInProgress	shared/contracts.py	Idempotency sentinel
Heartbeat	shared/contracts.py	Agent → Registry
ArtifactManifest	shared/contracts.py	Attached to tasks and results
AgentID	shared/contracts.py	ID parsing utility
ProviderAdapter	gateway/credentials/providers.py	Base class for OAuth provider adapters (§22.6)

Note: Credential data (accounts, credentials, resource_bindings, audit) is stored in SQLite tables in gateway/credentials.db , not as Pydantic models. The input.auth dict in TaskEnvelopes is a convention on the existing input: dict field (see §7.3), not a separate model.

Appendix C: Quick Reference — Event Type Enum

Event	Terminal?	Triggers
task.accepted	No	Agent received and validated task
task.progress	No	Intermediate progress update
task.suspended	No	Agent waiting for reverse-task reply
task.resumed	No	Agent continuing after suspension
task.deferred	No	Another instance executing — orchestrator owns recovery timer
artifact.added	No	Artifact produced
task.rejected	No	Stale epoch — triggers orchestrator redrive
task.completed	Yes	Success — closes task state, expires seq key
task.failed	Yes	Failure — closes task state, expires seq key
task.dlq	Yes	Dead letter queue — closes task state, expires seq key